

Data Mining: Concepts and Techniques

Classification

Jayanti Dansana

Classification: Definition

- Given a collection of records (*training set*)
 - Each record contains a set of *attributes*, one of the attributes is the *class*.
- Find a *model* for class attribute as a function of the values of other attributes.
- Goal: previously unseen records should be assigned a class as accurately as possible.
 - A *test set* is used to determine the accuracy of the model. Usually, the given data set is divided into training and test sets, with training set used to build the model and test set used to validate it.

Bayes Classifier

- A probabilistic framework for solving classification problems
- **Conditional Probability:**

$$P(C | A) = \frac{P(A \cap C)}{P(A)}$$

$$P(A | C) = \frac{P(A \cap C)}{P(C)}$$

- Bayes theorem:

$$P(C | A) = \frac{P(A | C)P(C)}{P(A)}$$

Example of Bayes Theorem

- Given:
 - A doctor knows that **meningitis** causes **stiff neck** 50% of the time
 - Prior probability of any patient having **meningitis** is 1/50,000
 - Prior probability of any patient having **stiff neck** is 1/20
- If a patient has stiff neck, what's the probability he/she has meningitis?

$$P(M | S) = \frac{P(S | M)P(M)}{P(S)} = \frac{0.5 \times 1/50000}{1/20} = 0.0002$$

Bayesian Classifiers

- Consider each attribute and class label as random variables
- Given a record with attributes $(A_1, A_2, \dots, A_n)$
 - Goal is to predict class C
 - Specifically, we want to find the value of C that maximizes $P(C | A_1, A_2, \dots, A_n)$
- Can we estimate $P(C | A_1, A_2, \dots, A_n)$ directly from data?

Bayesian Classifiers

- Approach:
 - compute the posterior probability $P(C \mid A_1, A_2, \dots, A_n)$ for all values of C using the Bayes theorem

$$P(C \mid A_1 A_2 \dots A_n) = \frac{P(A_1 A_2 \dots A_n \mid C) P(C)}{P(A_1 A_2 \dots A_n)}$$

- Choose value of C that maximizes $P(C \mid A_1, A_2, \dots, A_n)$
 - Equivalent to choosing value of C that maximizes $P(A_1, A_2, \dots, A_n \mid C) P(C)$
- How to estimate $P(A_1, A_2, \dots, A_n \mid C)$?

Naïve Bayes Classifier

- Assume independence among attributes A_i when class is given:
 - $P(A_1, A_2, \dots, A_n | C) = P(A_1 | C_j) P(A_2 | C_j) \dots P(A_n | C_j)$
 - Can estimate $P(A_i | C_j)$ for all A_i and C_j .
 - New point is classified to C_j if $P(C_j) \prod P(A_i | C_j)$ is maximal.

How to Estimate Probabilities from Data?

<i>Tid</i>	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

- Class: $P(C) = N_c / N$
 - e.g., $P(\text{No}) = 7/10$,
 $P(\text{Yes}) = 3/10$

- For discrete attributes:

$$P(A_i | C_k) = |A_{ik}| / N_{ck}$$

- where $|A_{ik}|$ is number of instances having attribute A_i and belongs to class C_k
- Examples:

$$P(\text{Status}=\text{Married}|\text{No}) = 4/7$$
$$P(\text{Refund}=\text{Yes}|\text{Yes})=0$$

Example of Naïve Bayes Classifier

Given a Test Record:

$X = (\text{Refund} = \text{No}, \text{Married}, \text{Income} = 120\text{K})$

naive Bayes Classifier:

$P(\text{Refund}=\text{Yes}|\text{No}) = 3/7$
 $P(\text{Refund}=\text{No}|\text{No}) = 4/7$
 $P(\text{Refund}=\text{Yes}|\text{Yes}) = 0$
 $P(\text{Refund}=\text{No}|\text{Yes}) = 1$
 $P(\text{Marital Status}=\text{Single}|\text{No}) = 2/7$
 $P(\text{Marital Status}=\text{Divorced}|\text{No}) = 1/7$
 $P(\text{Marital Status}=\text{Married}|\text{No}) = 4/7$
 $P(\text{Marital Status}=\text{Single}|\text{Yes}) = 2/7$
 $P(\text{Marital Status}=\text{Divorced}|\text{Yes}) = 1/7$
 $P(\text{Marital Status}=\text{Married}|\text{Yes}) = 0$

For taxable income:

If class=No: sample mean=110
 sample variance=2975
If class=Yes: sample mean=90
 sample variance=25

- $P(X|\text{Class}=\text{No}) = P(\text{Refund}=\text{No}|\text{Class}=\text{No})$
 $\times P(\text{Married}|\text{Class}=\text{No})$
 $\times P(\text{Income}=120\text{K}|\text{Class}=\text{No})$
 $= 4/7 \times 4/7 \times 0.0072 = 0.0024$
- $P(X|\text{Class}=\text{Yes}) = P(\text{Refund}=\text{No}|\text{Class}=\text{Yes})$
 $\times P(\text{Married}|\text{Class}=\text{Yes})$
 $\times P(\text{Income}=120\text{K}|\text{Class}=\text{Yes})$
 $= 1 \times 0 \times 1.2 \times 10^{-9} = 0$

Since $P(X|\text{No})P(\text{No}) > P(X|\text{Yes})P(\text{Yes})$

Therefore $P(\text{No}|X) > P(\text{Yes}|X)$

$\Rightarrow \text{Class} = \text{No}$

Naïve Bayes Classifier: Training Dataset

Class:

C_1 :buys_computer = 'yes'

C_2 :buys_computer = 'no'

Data to be classified:

X = (age $\leq$ 30,

Income = medium,

Student = yes

Credit_rating = Fair)

age	income	student	credit ratings	comp
≤ 30	high	no	fair	no
≤ 30	high	no	excellent	no
31...40	high	no	fair	yes
> 40	medium	no	fair	yes
> 40	low	yes	fair	yes
> 40	low	yes	excellent	no
31...40	low	yes	excellent	yes
≤ 30	medium	no	fair	no
≤ 30	low	yes	fair	yes
> 40	medium	yes	fair	yes
≤ 30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
> 40	medium	no	excellent	no

Naïve Bayes Classifier: An Example

- $P(C_i)$: $P(\text{buys_computer} = \text{"yes"}) = 9/14 = 0.643$

$$P(\text{buys_computer} = \text{"no"}) = 5/14 = 0.357$$

- Compute $P(X|C_i)$ for each class

$$P(\text{age} = \text{"<=30"} \mid \text{buys_computer} = \text{"yes"}) = 2/9 = 0.222$$

$$P(\text{age} = \text{"<= 30"} \mid \text{buys_computer} = \text{"no"}) = 3/5 = 0.6$$

$$P(\text{income} = \text{"medium"} \mid \text{buys_computer} = \text{"yes"}) = 4/9 = 0.444$$

$$P(\text{income} = \text{"medium"} \mid \text{buys_computer} = \text{"no"}) = 2/5 = 0.4$$

$$P(\text{student} = \text{"yes"} \mid \text{buys_computer} = \text{"yes"}) = 6/9 = 0.667$$

$$P(\text{student} = \text{"yes"} \mid \text{buys_computer} = \text{"no"}) = 1/5 = 0.2$$

$$P(\text{credit_rating} = \text{"fair"} \mid \text{buys_computer} = \text{"yes"}) = 6/9 = 0.667$$

$$P(\text{credit_rating} = \text{"fair"} \mid \text{buys_computer} = \text{"no"}) = 2/5 = 0.4$$

- $X = (\text{age} \leq 30, \text{income} = \text{medium}, \text{student} = \text{yes}, \text{credit_rating} = \text{fair})$

$$P(X|C_i) : P(X|\text{buys_computer} = \text{"yes"}) = 0.222 \times 0.444 \times 0.667 \times 0.667 = 0.044$$

$$P(X|\text{buys_computer} = \text{"no"}) = 0.6 \times 0.4 \times 0.2 \times 0.4 = 0.019$$

$$P(X|C_i) * P(C_i) : P(X|\text{buys_computer} = \text{"yes"}) * P(\text{buys_computer} = \text{"yes"}) = 0.028$$

$$P(X|\text{buys_computer} = \text{"no"}) * P(\text{buys_computer} = \text{"no"}) = 0.007$$

age	income	student	credit rating	comp
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Therefore, X belongs to class ("buys_computer = yes")

Naïve Bayes Classifier: Comments

- Advantages
 - Easy to implement
 - Good results obtained in most of the cases
- Disadvantages
 - Assumption: class conditional independence, therefore loss of accuracy
 - Practically, dependencies exist among variables
 - E.g., hospitals: patients: Profile: age, family history, etc.
Symptoms: fever, cough etc., Disease: lung cancer, diabetes, etc.
 - Dependencies among these cannot be modeled by Naïve Bayes Classifier
- How to deal with these dependencies? Bayesian Belief Networks

Rule-Based Classifier

- Classify records by using a collection of “if...then...” rules
- Rule: $(Condition) \rightarrow y$
 - where
 - *Condition* is a conjunctions of attributes
 - *y* is the class label
 - *LHS*: rule antecedent or condition
 - *RHS*: rule consequent
 - Examples of classification rules:
 - $(\text{Blood Type}=\text{Warm}) \wedge (\text{Lay Eggs}=\text{Yes}) \rightarrow \text{Birds}$
 - $(\text{Taxable Income} < 50\text{K}) \wedge (\text{Refund}=\text{Yes}) \rightarrow \text{Evade}=\text{No}$

Rule-based Classifier: Example

Name	Blood Type	Give Birth	Can Fly	Live in Water	Class
human	warm	yes	no	no	mammals
python	cold	no	no	no	reptiles
salmon	cold	no	no	yes	fishes
whale	warm	yes	no	yes	mammals
frog	cold	no	no	sometimes	amphibians
komodo	cold	no	no	no	reptiles
bat	warm	yes	yes	no	mammals
pigeon	warm	no	yes	no	birds
cat	warm	yes	no	no	mammals
leopard shark	cold	yes	no	yes	fishes
turtle	cold	no	no	sometimes	reptiles
penguin	warm	no	no	sometimes	birds
porcupine	warm	yes	no	no	mammals
eel	cold	no	no	yes	fishes
salamander	cold	no	no	sometimes	amphibians
gila monster	cold	no	no	no	reptiles
platypus	warm	no	no	no	mammals
owl	warm	no	yes	no	birds
dolphin	warm	yes	no	yes	mammals
eagle	warm	no	yes	no	birds

R1: (Give Birth = no) $\wedge$ (Can Fly = yes) $\rightarrow$ Birds

R2: (Give Birth = no) $\wedge$ (Live in Water = yes) $\rightarrow$ Fishes

R3: (Give Birth = yes) $\wedge$ (Blood Type = warm) $\rightarrow$ Mammals

R4: (Give Birth = no) $\wedge$ (Can Fly = no) $\rightarrow$ Reptiles

R5: (Live in Water = sometimes) $\rightarrow$ Amphibians

Application of Rule-Based Classifier

- A rule r **covers** an instance x if the attributes of the instance satisfy the condition of the rule

R1: (Give Birth = no) $\wedge$ (Can Fly = yes) $\rightarrow$ Birds

R2: (Give Birth = no) $\wedge$ (Live in Water = yes) $\rightarrow$ Fishes

R3: (Give Birth = yes) $\wedge$ (Blood Type = warm) $\rightarrow$ Mammals

R4: (Give Birth = no) $\wedge$ (Can Fly = no) $\rightarrow$ Reptiles

R5: (Live in Water = sometimes) $\rightarrow$ Amphibians

Name	Blood Type	Give Birth	Can Fly	Live in Water	Class
hawk	warm	no	yes	no	?
grizzly bear	warm	yes	no	no	?

The rule R1 covers a hawk $\Rightarrow$ Bird

The rule R3 covers the grizzly bear $\Rightarrow$ Mammal

Rule Coverage and Accuracy

- Coverage of a rule:
 - Fraction of records that satisfy the antecedent of a rule
- Accuracy of a rule:
 - Fraction of records that satisfy both the antecedent and consequent of a rule

<i>Tid</i>	Refund	Marital Status	Taxable Income	Class
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

(Status=Single) $\rightarrow$ No

Coverage = 40%, Accuracy = 50%

How does Rule-based Classifier Work?

R1: (Give Birth = no) $\wedge$ (Can Fly = yes) $\rightarrow$ Birds

R2: (Give Birth = no) $\wedge$ (Live in Water = yes) $\rightarrow$ Fishes

R3: (Give Birth = yes) $\wedge$ (Blood Type = warm) $\rightarrow$ Mammals

R4: (Give Birth = no) $\wedge$ (Can Fly = no) $\rightarrow$ Reptiles

R5: (Live in Water = sometimes) $\rightarrow$ Amphibians

Name	Blood Type	Give Birth	Can Fly	Live in Water	Class
lemur	warm	yes	no	no	?
turtle	cold	no	no	sometimes	?
dogfish shark	cold	yes	no	yes	?

A lemur triggers rule R3, so it is classified as a mammal

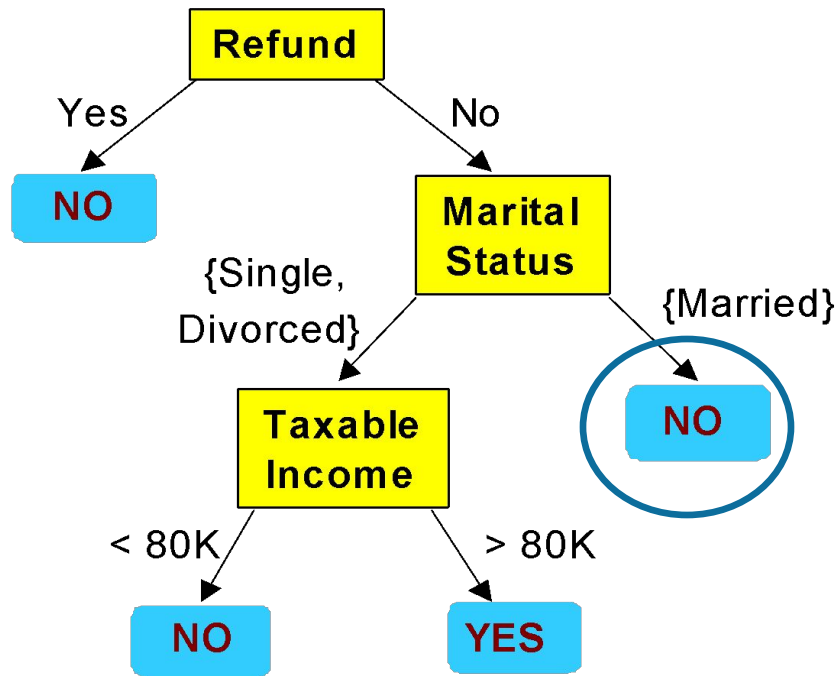
A turtle triggers both R4 and R5

A dogfish shark triggers none of the rules

Characteristics of Rule-Based Classifier

- Mutually exclusive rules
 - Classifier contains mutually exclusive rules if the rules are independent of each other
 - Every record is covered by at most one rule
- Exhaustive rules
 - Classifier has exhaustive coverage if it accounts for every possible combination of attribute values
 - Each record is covered by at least one rule

Rules Can Be Simplified



Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Initial Rule: $(\text{Refund}=\text{No}) \wedge (\text{Status}=\text{Married}) \rightarrow \text{No}$

Simplified Rule: $(\text{Status}=\text{Married}) \rightarrow \text{No}$

Classification by Backpropagation

- **Backpropagation**: A neural network learning algorithm
- Biologically motivated approach to machine learning

Similarity with biological network

- Fundamental processing elements of a neural network is a neuron
 1. Receives inputs from other source
 2. Combines them in someway
 3. Performs a generally nonlinear operation on the result
 4. Outputs the final result

Neural Network

- **Neural Network** is a set of connected INPUT/OUTPUT UNITS, where each connection has a WEIGHT associated with it.
- **Neural Network** learning is also called CONNECTIONIST learning due to the connections between units.
- It is a case of **SUPERVISED**, **INDUCTIVE** or **CLASSIFICATION** learning.

Neural Network

- **Neural Network** learns by adjusting the weights so as to be able to correctly classify the training data and hence, after testing phase, to classify unknown data
- **Neural Network** needs long time for training
- **Neural Network** has a high tolerance to noisy and incomplete data

Neural Network as a Classifier

- Weakness
 - long training time
 - difficult to understand the learned function (weights)
 - not easy to incorporate domain knowledge
 - Require a number of parameters typically best determined empirically, e.g., the network topology or “structure”
 - **Poor interpretability**: Difficult to interpret the symbolic meaning behind the learned weights and of “hidden units” in the network
- Strength
 - High tolerance to noisy data
 - Ability to classify untrained patterns
 - Well-suited for continuous-valued inputs and outputs
 - Algorithms are inherently parallel

Neural Network Classifier

- **Input: Classification data**

It contains classification attribute

- Data is divided, as in any classification problem.

[Training data and Testing data]

- All data must be normalized

(i.e. all values of attributes in the database are changed to contain values in the interval $[0,1]$ or $[-1,1]$)

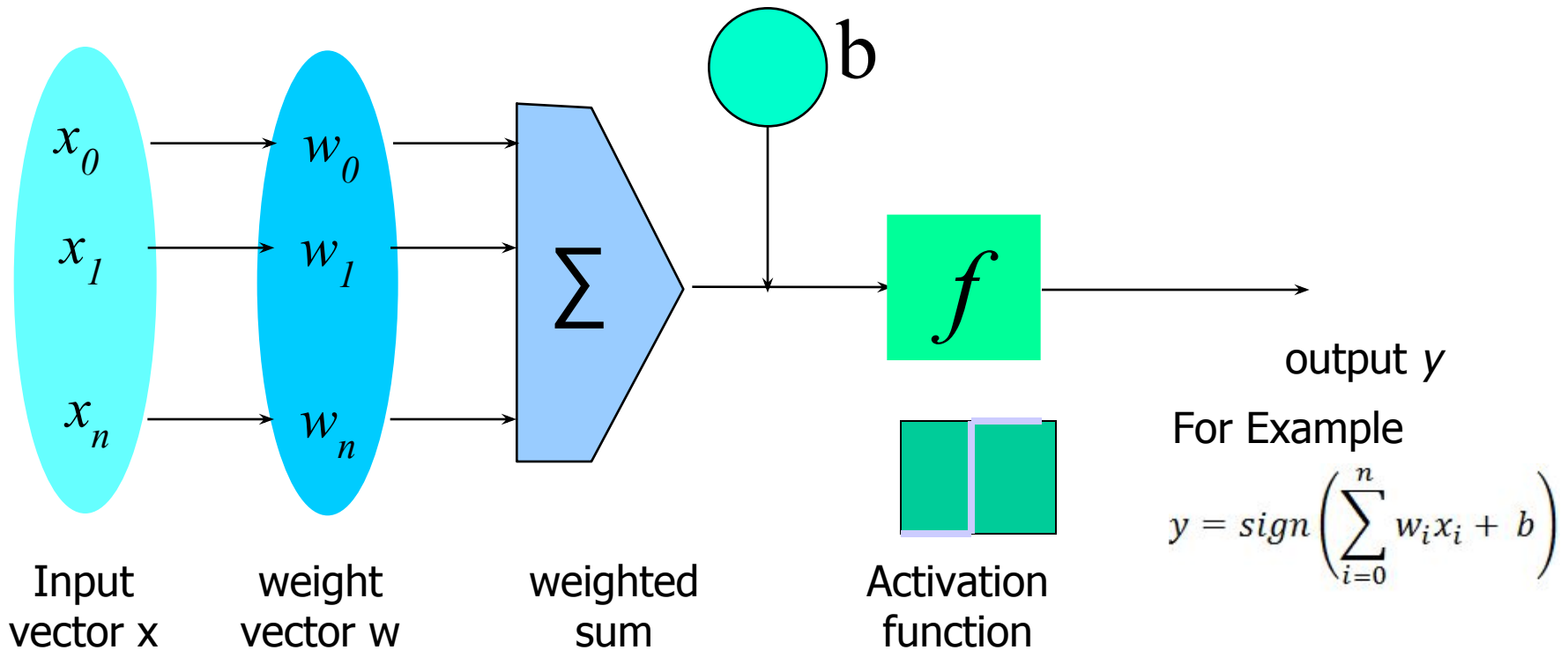
Neural Network can work with data in the range of $(0, 1)$ or $(-1, 1)$

- **Two basic normalization techniques**

[1] Max-Min normalization

[2] Decimal Scaling normalization

A Neuron (= a perceptron)



- The n -dimensional input vector x is mapped into variable y by means of the scalar product and a nonlinear function mapping

One Neuron as a Network

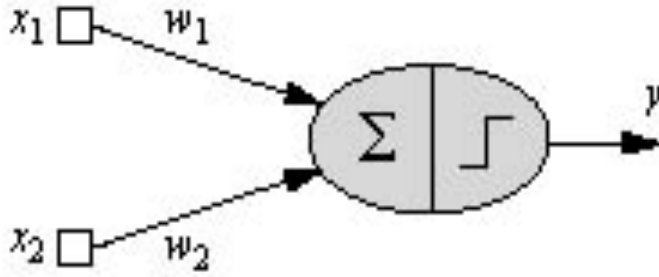


Fig1: an artificial neuron

- Here x_1 and x_2 are normalized attribute value of data.
- y is the output of the neuron , i.e the class label.
- x_1 and x_2 values multiplied by weight values w_1 and w_2 are input to the neuron x .
- Value of x_1 is multiplied by a weight w_1 and values of x_2 is multiplied by a weight w_2 .
- Given that
 - $w_1 = 0.5$ and $w_2 = 0.5$
 - Say, value of x_1 is 0.3 and value of x_2 is 0.8,
 - So, weighted sum is :
 - $\text{sum} = w_1 \times x_1 + w_2 \times x_2 = 0.5 \times 0.3 + 0.5 \times 0.8 = 0.55$

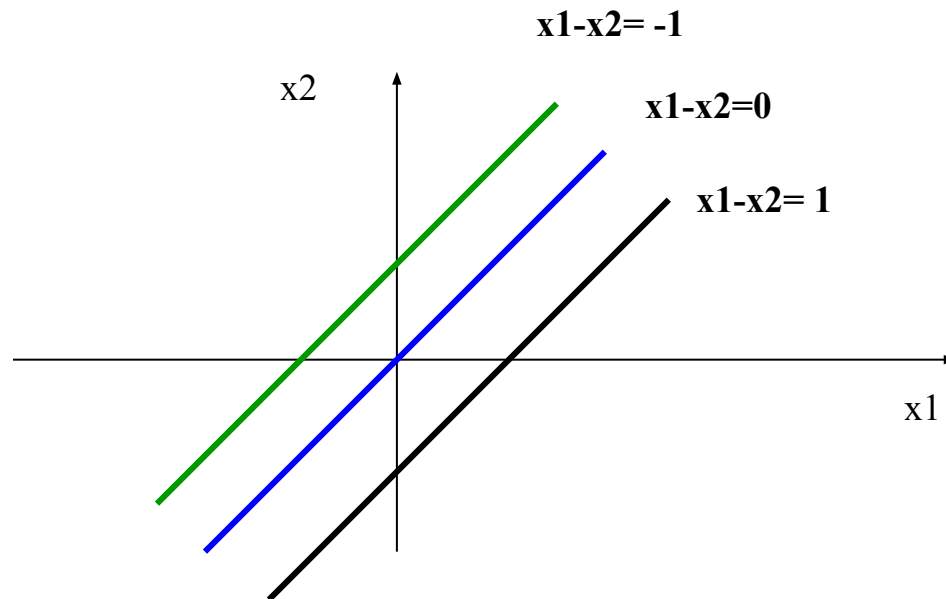
One Neuron as a Network

- The neuron receives the weighted sum as input and calculates the output as a function of input as follows :
 - $y = f(x)$, where $f(x)$ is defined as
 - $f(x) = 0$ { when $x < 0.5$ }
 - $f(x) = 1$ { when $x \geq 0.5$ }
- For our example, x (weighted sum) is 0.55, so $y = 1$,
- That means corresponding input attribute values are classified in class 1.
- If for another input values , $x = 0.45$, then $f(x) = 0$,
- so we could conclude that input values are classified to class 0.

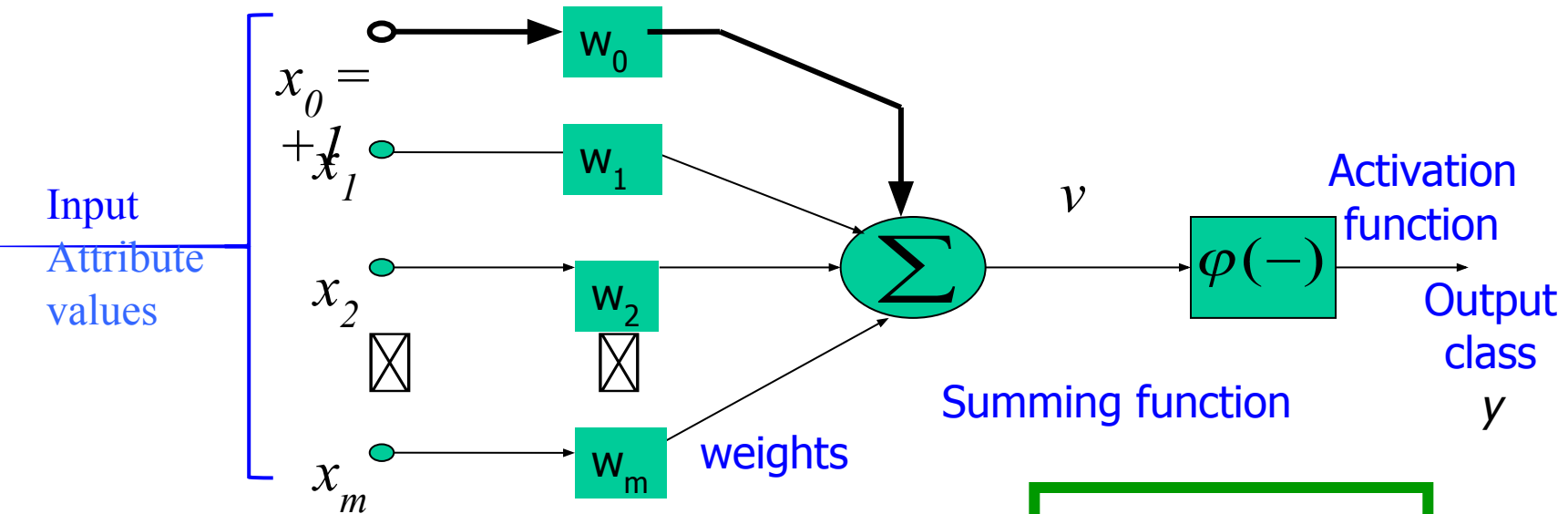
Bias of a Neuron

- We need the bias value to be added to the weighted sum $\sum w_i x_i$ so that we can transform it from the origin.

$$v = \sum w_i x_i + b, \text{ here } b \text{ is the bias}$$



Bias as extra input



$$v = \sum_{j=0}^m w_j x_j$$
$$w_0 = b$$

Neuron with Activation

- The neuron is the basic information processing unit of a NN. It consists of:
 1. A set of **links**, describing the neuron inputs, with **weights** $w_1, w_2, \dots, w_m$
 2. An **adder** function (linear combiner) for computing the weighted sum of the inputs (real numbers):

$$v = \sum_{i=0}^m w_i x_i$$

3. **Activation function** : for limiting the amplitude of the neuron output.

$$y = f(v + b)$$

Neural Network Learning

- The inputs are fed simultaneously into the input layer.
- The weighted outputs of these units are fed into **hidden layer**.
- The weighted outputs of the last hidden layer are inputs to units making up the output layer.

A Multilayer Feed Forward Network

- The units in the hidden layers and output layer are sometimes referred to as **neurodes**, due to their symbolic biological basis, or as **output units**.
- A network containing two hidden layers is called a **three-layer** neural network, and so on.
- The network is feed-forward in that none of the weights cycles back to an input unit or to an output unit of a previous layer.

A Multilayered Feed – Forward Network

- **Input:** records without class attribute with normalized attributes values.
- **Input Vector:** $X = \{x_1, x_2, \dots, x_n\}$, where n is the number of (non class) attributes.
- **Input Layer** – there are as many nodes as non-class attributes i.e. as the length of the input vector.
- **Hidden Layer** – the number of nodes in the hidden layer and the number of hidden layers depends on implementation.

A Multilayered Feed-Forward Network

- **Output Layer** – corresponds to the class attribute.
- There are as many nodes as classes (values of the class attribute).

$$O_k$$

$$k = 1, 2, \dots \text{\#classes}$$

- Network is **fully connected**, i.e. each unit provides input to each unit in the next forward layer.

Classification by Back propagation

- **Back Propagation** learns by iteratively processing a set of training data (samples).
- For each sample, weights are modified to minimize the error between network's classification and actual classification.

Steps in Back propagation Algorithm

- **Step One:** initialize the weights and biases.
- The weights in the network are initialized to random numbers from the interval $[-1, 1]$.
- Each unit has a BIAS associated with it
- The biases are similarly initialized to random numbers from the interval $[-1, 1]$.
- **Step Two:** feed the training sample.

Propagate the inputs forward

- For unit j in the input layer, its output is equal to its input, that is, $O_j = I_j$

for input unit j .

- The net input to each unit in the hidden and output layers is computed as follows.
- Given a unit j in a hidden or output layer, the net input is

$$I_j = \sum_i w_{ij} O_i + \theta_j$$

where w_{ij} is the weight of the connection from unit i in the previous layer to unit j ; O_i is the output of unit i from the previous layer; θ_j is the bias of the unit.

Propagate the inputs forward

- Each unit in the hidden and output layers takes its net input and then applies an **activation function**. The function symbolizes the activation of the neuron represented by the unit. It is also called a **logistic**, **sigmoid**, or **squashing function**.
- Given a net input I_j to unit j , then

$$O_j = f(I_j),$$

the output of unit j , is computed as

$$O_j = \frac{1}{1 + e^{-I_j}}$$

Back propagate the error

- When reaching the Output layer, the error is computed and propagated backwards.
- For a unit k in the output layer the error is computed by a formula:

$$Err_k = O_k(1 - O_k)(T_k - O_k)$$

where O_k – actual output of unit k (computed by activation function.

$$O_k = \frac{1}{1 + e^{-I_k}}$$

T_k – True output based of known class label; classification of training sample

$O_k(1-O_k)$ – is a Derivative (rate of change) of activation function.

Back propagate the error

- The error is propagated backwards by updating weights and biases to reflect the error of the network classification .
- For a unit j in the hidden layer the error is computed by a formula:

$$Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$$

where w_{jk} is the weight of the connection from unit j to unit k in the next higher layer, and Err_k is the error of unit k .

Update weights and biases

- **Weights** are updated by the following equations, where l is a constant between 0.0 and 1.0 reflecting **the learning rate**, this learning rate is **fixed for implementation**.

$$\Delta w_{ij} = (l)Err_j O_i$$

$$w_{ij} = w_{ij} + \Delta w_{ij}$$

- **Biases** are updated by the following equations

$$\Delta \theta_j = (l)Err_j$$

$$\theta_j = \theta_j + \Delta \theta_j$$

Update weights and biases

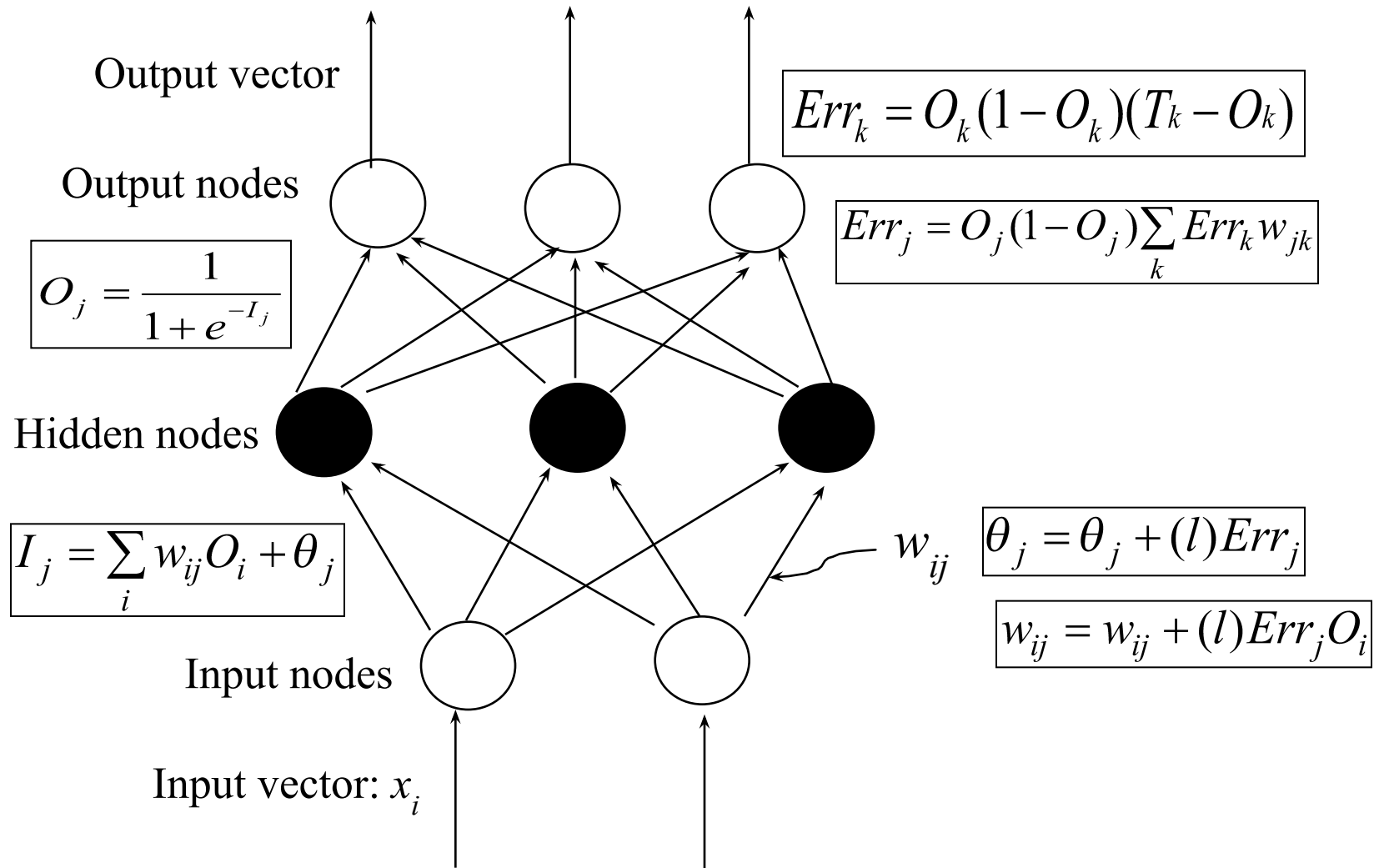
- We are updating weights and biases after the presentation of each sample.
- This is called case updating.
- Epoch --- One iteration through the training set is called an epoch.
- Epoch updating -----
- Alternatively, the weight and bias increments could be accumulated in variables and the weights and biases updated after all of the samples of the training set have been presented.
- Case updating is more accurate

Terminating Conditions

- Training stops

- All Δw_{ij} in the previous epoch are below some threshold, or
- The percentage of samples misclassified in the previous epoch is below some threshold, or
- a pre specified number of epochs has expired.
- In practice, several hundreds of thousands of epochs may be required before the weights will converge.

Backpropagation Formulas



Example of Back propagation

Input = 3,

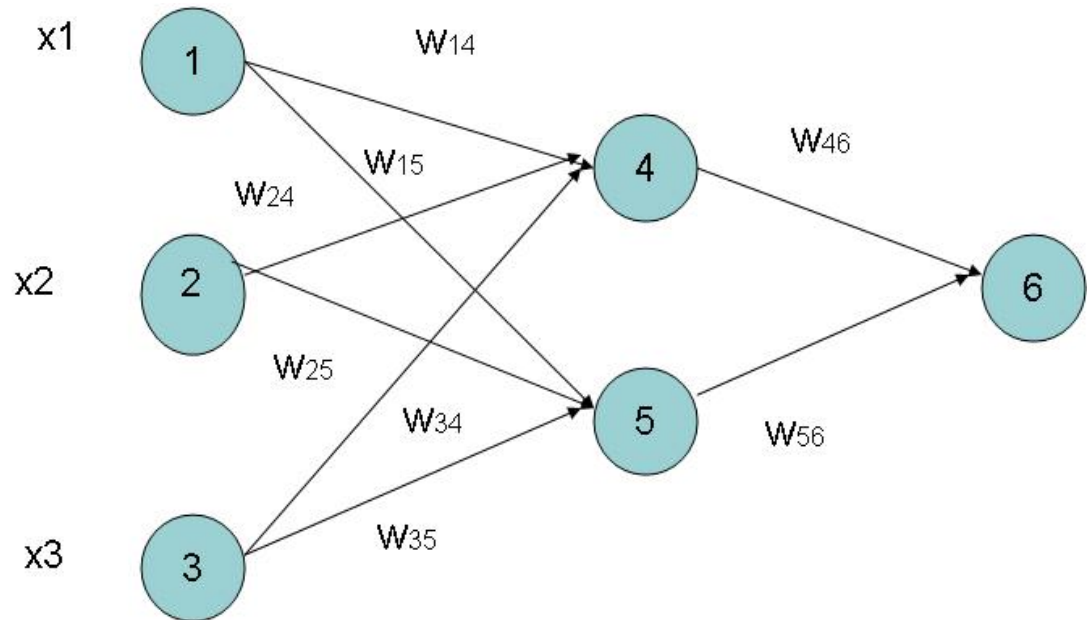
Hidden Neuron = 2

Output = 1

Initialize weights :

Random Numbers from
-1.0 to 1.0

Initial Input and weight

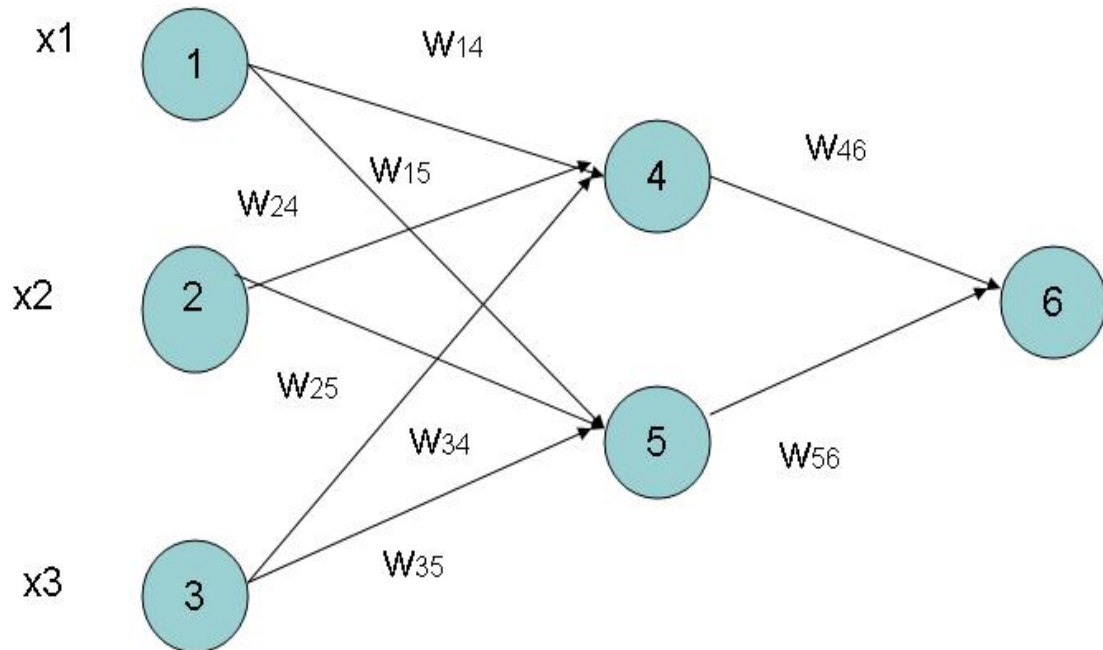


x_1	x_2	x_3	w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}
1	0	1	0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2

Example (Contd...)

- Bias added to Hidden + Output nodes
- Initialize Bias
- Random Values from -1.0 to 1.0
- Bias (Random)

θ_4	θ_5	θ_6
-0.4	0.2	0.1



Net Input and Output Calculation

Unit _j	Net Input I _j	Output O _j
4	$0.2 + 0 - 0.5 - 0.4 = -0.7$	$O_j = \frac{1}{1 + e^{0.7}} = 0.332$
5	$-0.3 + 0 + 0.2 + 0.2 = 0.1$	$O_j = \frac{1}{1 + e^{-0.1}} = 0.525$
6	$(-0.3)0.332 - (0.2)(0.525) + 0.1 = -0.105$	$O_j = \frac{1}{1 + e^{0.105}} = 0.475$

Calculation of Error at Each Node

Unit _j	Error _j
6	$0.475 \times (1-0.475) \times (1-0.475) = 0.1311$ We assume $T_6 = 1$
5	$0.525 \times (1-0.525) \times 0.1311 \times (-0.2) = 0.0065$
4	$0.332 \times (1-0.332) \times 0.1311 \times (-0.3) = -0.0087$

Calculation of weights and Bias Updating

Learning Rate $\eta = 0.9$

Weight	New Values
w_{46}	$-0.3 + 0.9(0.1311)(0.332) = -0.261$
w_{56}	$-0.2 + (0.9)(0.1311)(0.525) = -0.138$
w_{14}	$0.2 + 0.9(-0.0087)(1) = 0.192$
w_{15}	$-0.3 + (0.9)(-0.0065)(1) = -0.306$
.....similarly	similarly
θ_6	$0.1 + (0.9)(0.1311) = 0.218$
.....similarly	similarly

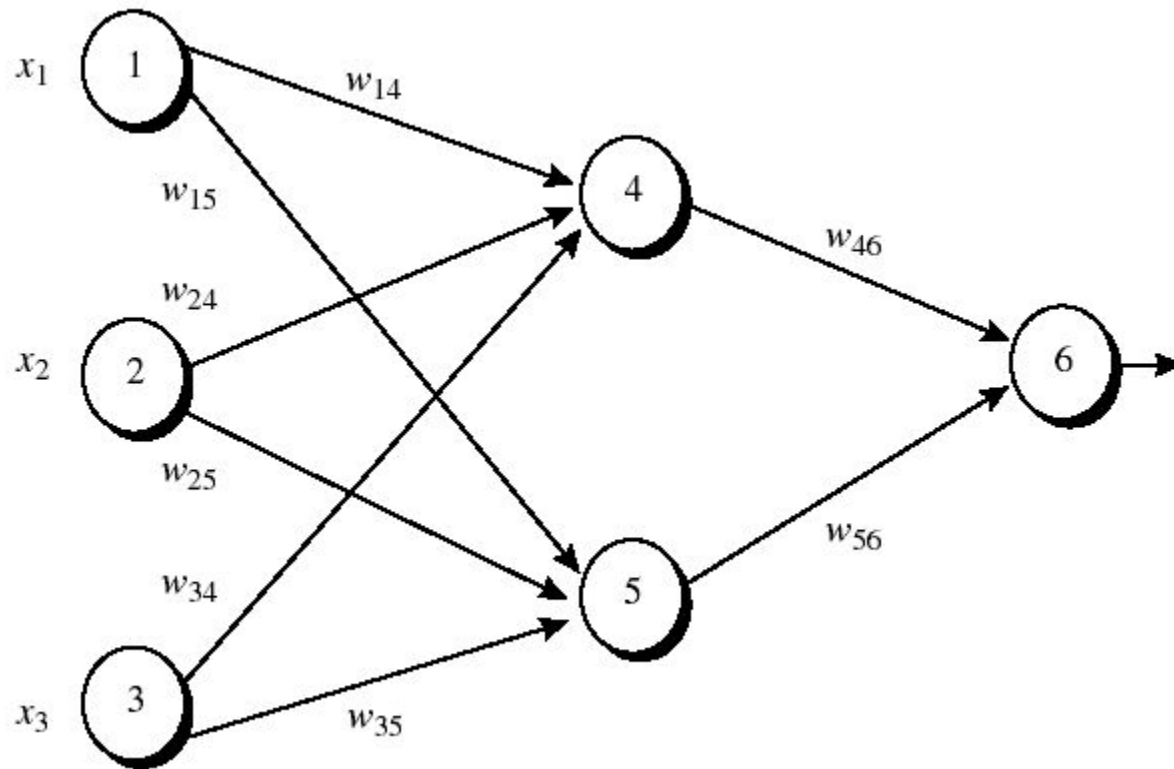


Figure 1: An example of a multilayer feed-forward neural network. Assume that the learning rate η is 0.9 and the first training example, $X = (1,0,1)$ whose class label is 1.

Note: The sigmoid function is applied to hidden layer and output layer. 50

Table 1: Initial input and weight values

x_1	x_2	x_3	w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	w_{04}	w_{05}	w_{06}
1	0	1	0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1

Table 2: The net input and output calculation

Unit j	Net input I_j	Output O_j
4	$0.2 + 0 - 0.5 - 0.4 = -0.7$	$1/(1+e^{0.7})=0.332$
5	$-0.3 + 0 + 0.2 + 0.2 = 0.1$	$1/(1+e^{0.1})=0.525$
6	$(-0.3)(0.332) - (0.2)(0.525) + 0.1 = -0.105$	$1/(1+e^{0.105})=0.474$

Table 3: Calculation of the error at each node

Unit j	δ_j
6	$(0.474)(1-0.474)(1-0.474)=0.1311$
5	$(0.525)(1-0.525)(0.1311)(-0.2)=-0.0065$
4	$(0.332)(1-0.332)(0.1311)(-0.3)=-0.0087$

Table 4: Calculation for weight updating

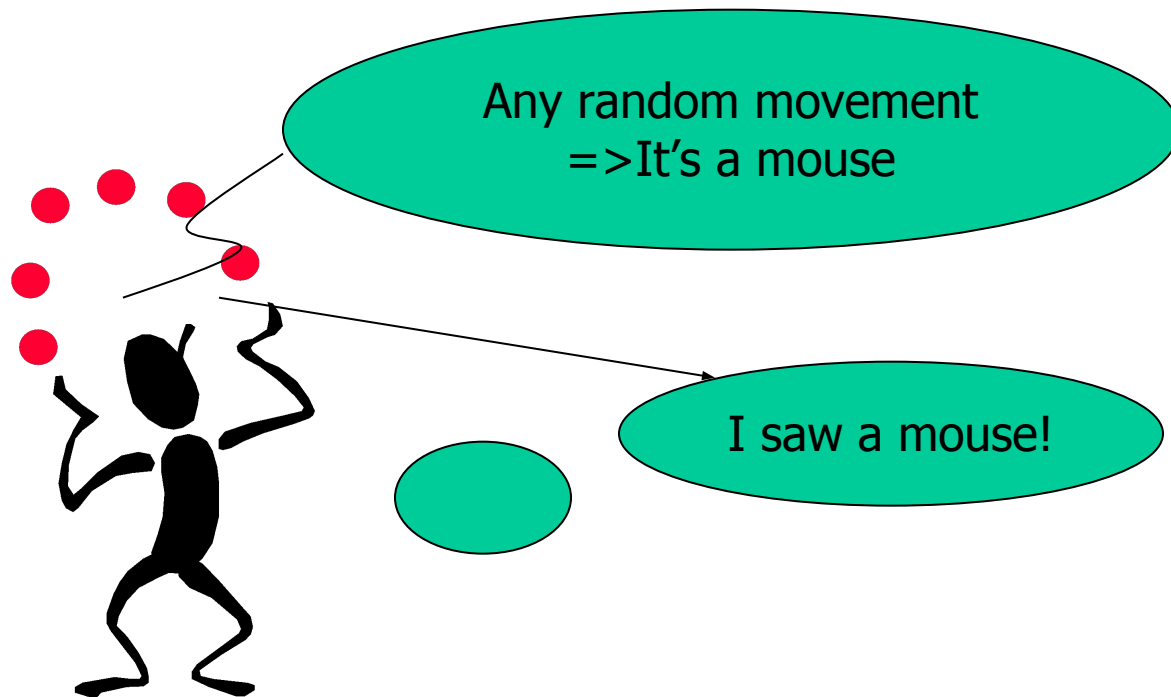
Weight	New value
w_{46}	$-0.3 + (0.9)(0.1311)(0.332) = -0.261$
w_{56}	$-0.2 + (0.9)(0.1311)(0.525) = -0.138$
w_{14}	$0.2 + (0.9)(-0.0087)(1) = 0.192$
w_{15}	$-0.3 + (0.9)(-0.0065)(1) = -0.306$
w_{24}	$0.4 + (0.9)(-0.0087)(0) = 0.4$
w_{25}	$0.1 + (0.9)(-0.0065)(0) = 0.1$
w_{34}	$-0.5 + (0.9)(-0.0087)(1) = -0.508$
w_{35}	$0.2 + (0.9)(-0.0065)(1) = 0.194$
w_{06}	$0.1 + (0.9)(0.1311) = 0.218$
w_{05}	$0.2 + (0.9)(-0.0065) = 0.194$
w_{04}	$-0.4 + (0.9)(-0.0087) = -0.408$

Different Learning Methods

- Eager Learning
 - Explicit description of target function on the whole training set
- Instance-based Learning
 - Learning=storing all training instances
 - Classification=assigning target function to a new instance
 - Referred to as “Lazy” learning

Different Learning Methods

- Eager Learning



Instance-based Learning



Instance-Based Classifiers

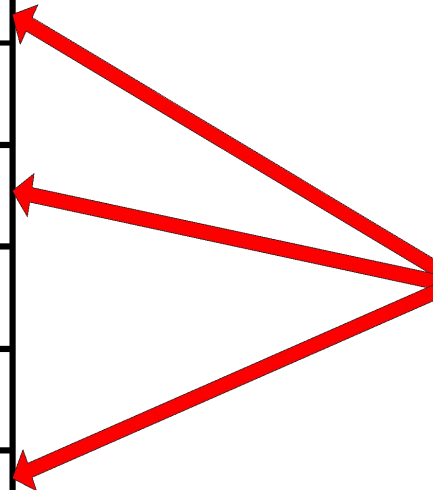
Set of Stored Cases

Atr1		AtrN	Class
			A
			B
			B
			C
			A
			C
			B

- Store the training records
- Use training records to predict the class label of unseen cases

Unseen Case

Atr1		AtrN

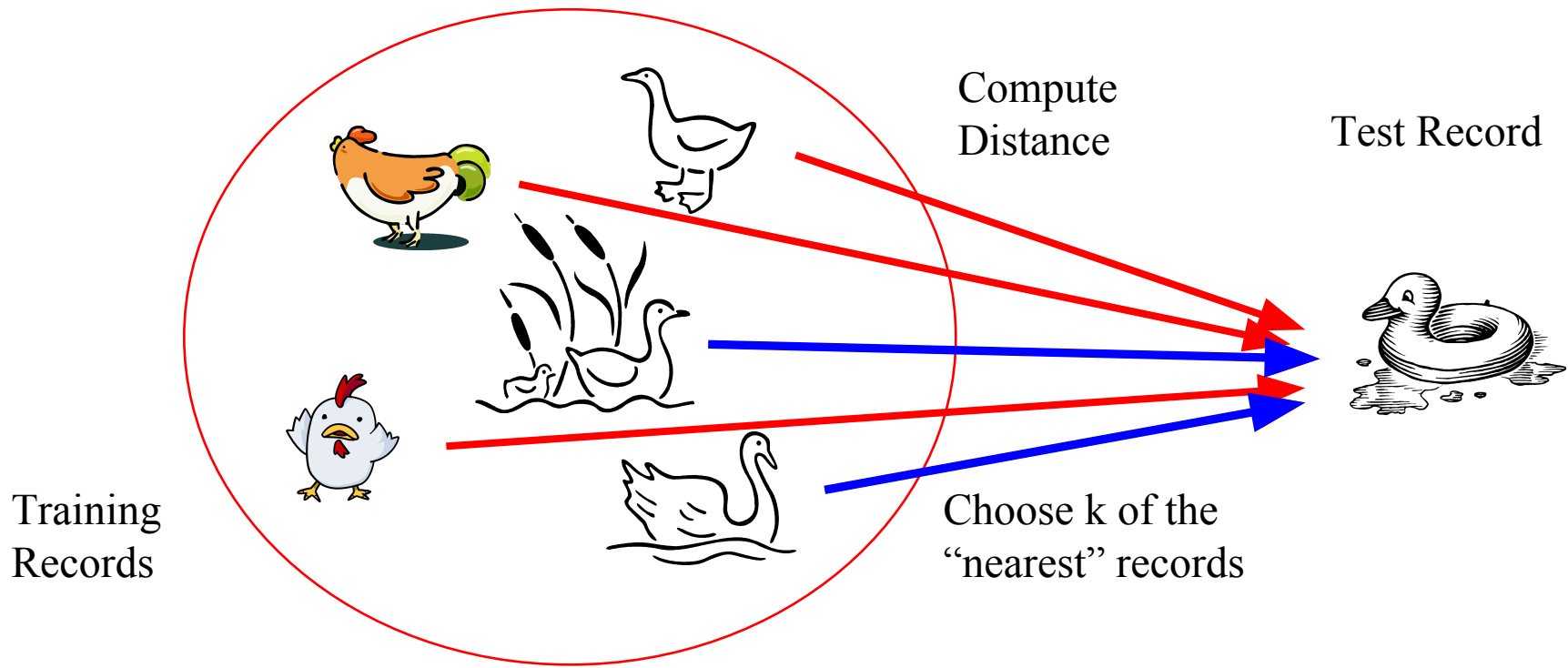


Instance Based Classifiers

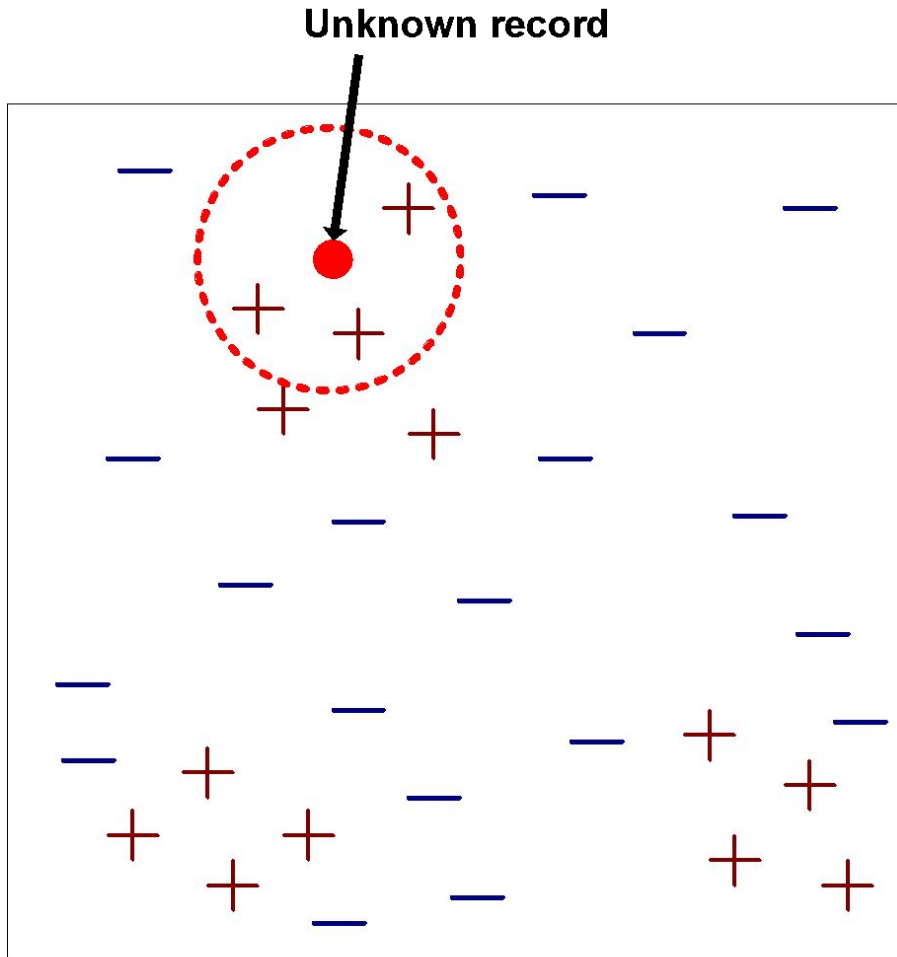
- Examples:
 - Rote-learner
 - Memorizes entire training data and performs classification only if attributes of record match one of the training examples exactly
 - Nearest neighbor
 - Uses k “closest” points (nearest neighbors) for performing classification

Nearest Neighbor Classifiers

- Basic idea:
 - If it walks like a duck, quacks like a duck, then it's probably a duck

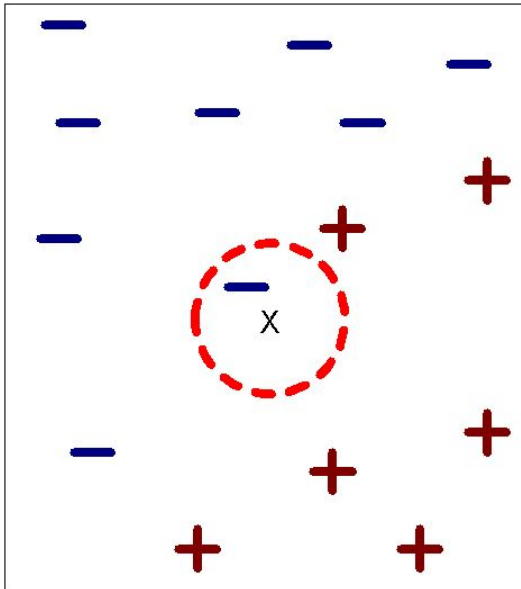


Nearest-Neighbor Classifiers

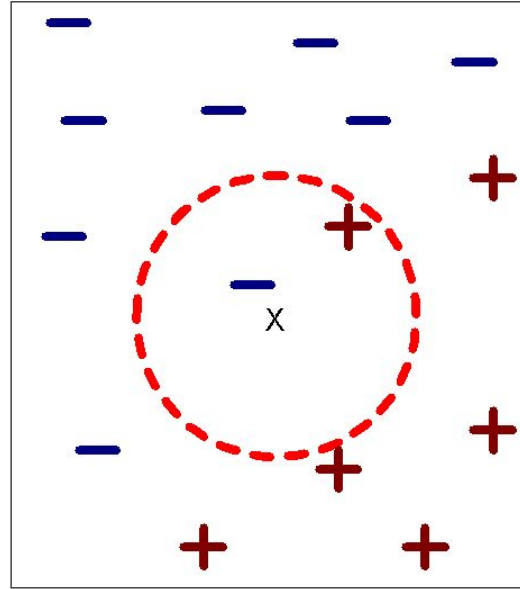


- Requires three things
 - The set of stored records
 - **Distance Metric** to compute distance between records
 - The value of **k , the number of nearest neighbors** to retrieve
- To classify an unknown record:
 - **Compute distance** to other training records
 - Identify **k** nearest neighbors
 - Use class labels of nearest neighbors to determine the class label of unknown record (e.g., by taking **majority vote**)

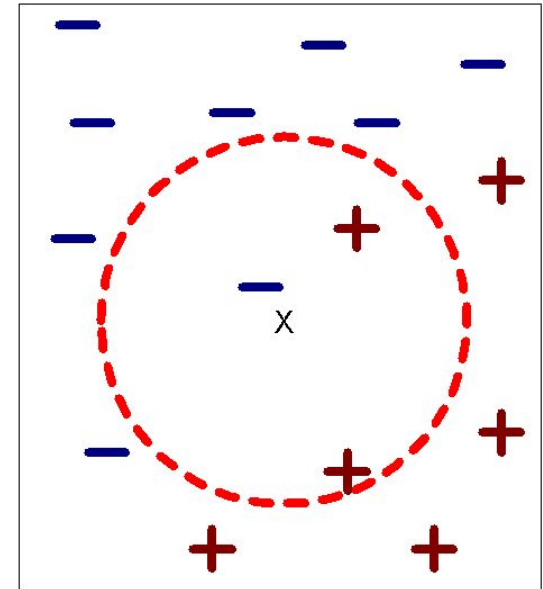
Definition of Nearest Neighbor



(a) 1-nearest neighbor



(b) 2-nearest neighbor

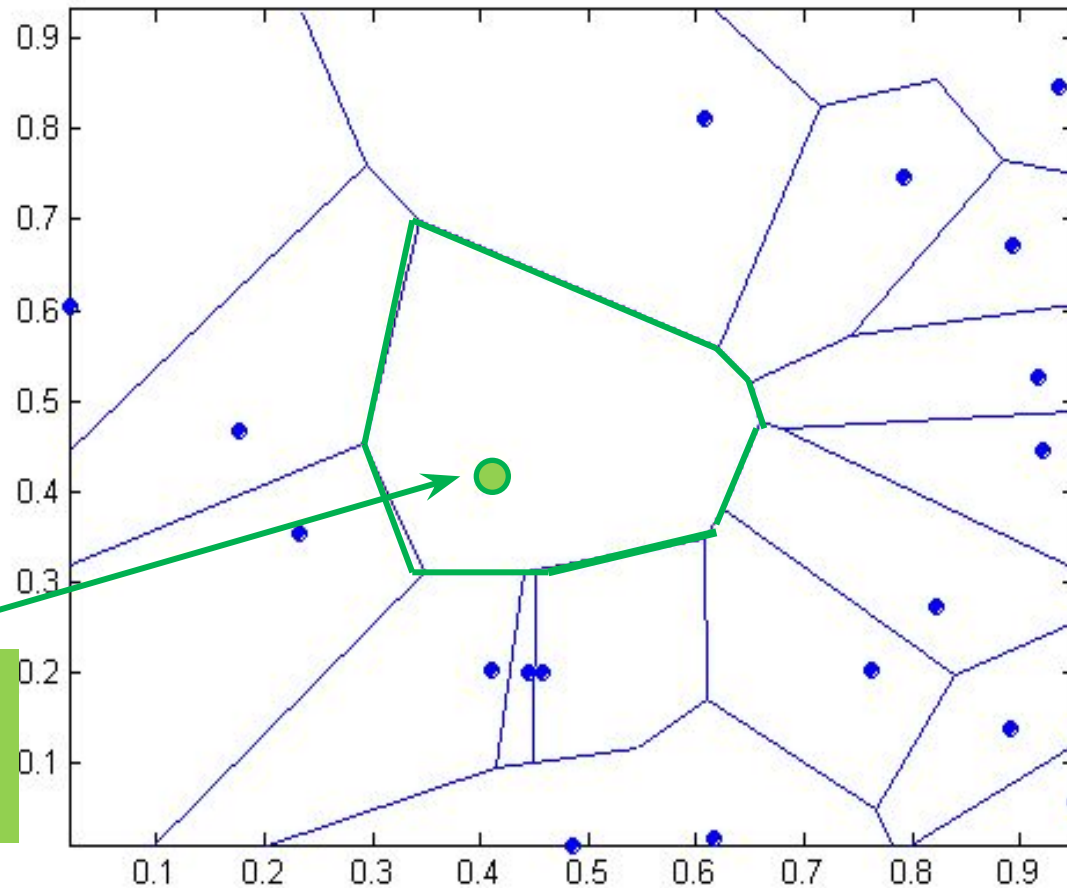


(c) 3-nearest neighbor

K-nearest neighbors of a record x are data points that have the k smallest distance to x

1 nearest-neighbor

Voronoi Diagram defines the classification boundary



The area takes the class of the green point

Nearest Neighbor Classification

- Compute distance between two points:

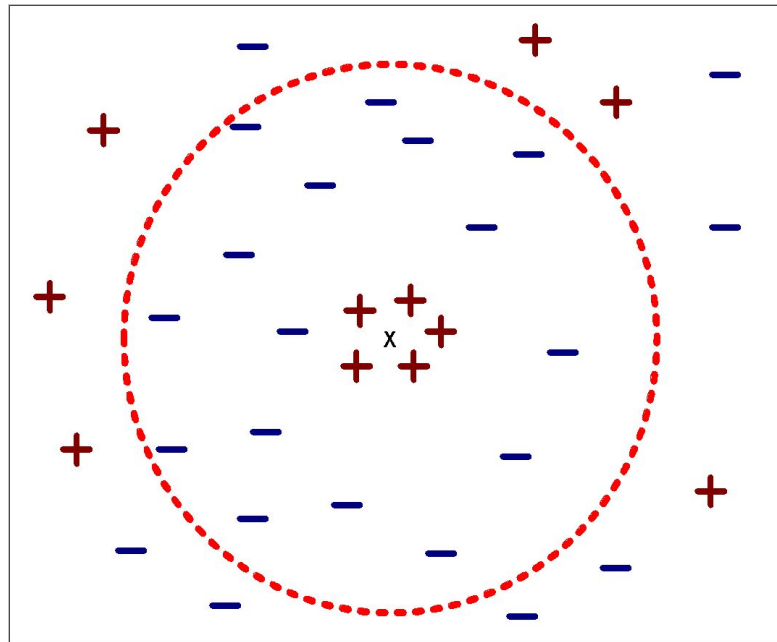
- Euclidean distance

$$d(p, q) = \sqrt{\sum_i (p_i - q_i)^2}$$

- Determine the class from nearest neighbor list
 - take the majority vote of class labels among the k-nearest neighbors
 - Weigh the vote according to distance
 - weight factor, $w = 1/d^2$

Nearest Neighbor Classification...

- Choosing the value of k :
 - If k is too small, sensitive to noise points
 - If k is too large, neighborhood may include points from other classes



Nearest Neighbor Classification...

- Scaling issues
 - Attributes may have to be scaled to prevent distance measures from being dominated by one of the attributes
 - Example:
 - height of a person may vary from 1.5m to 1.8m
 - weight of a person may vary from 90lb to 300lb
 - income of a person may vary from \$10K to \$1M

Nearest Neighbor Classification...

- Problem with Euclidean measure:
 - High dimensional data
 - **curse of dimensionality**
 - Can produce counter-intuitive results

1 1 1 1 1 1 1 1 1 1 1 0

0 1 1 1 1 1 1 1 1 1 1 1

$d = 1.4142$

VS

1 0 0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 0 1

$d = 1.4142$

Solution: Normalize the vectors to unit length

Nearest neighbor Classification...

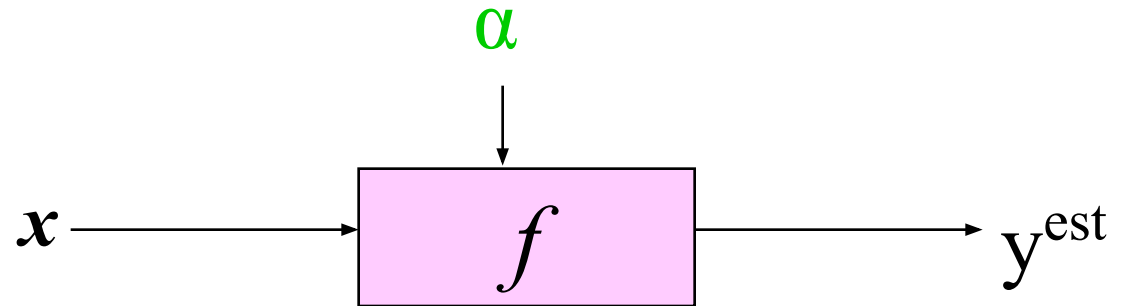
- k-NN classifiers are **lazy learners**
 - It does not build models explicitly
 - Unlike **eager learners** such as decision tree induction and rule-based systems
- Classifying unknown records are relatively expensive
 - Naïve algorithm: $O(n)$
 - Need for structures to retrieve nearest neighbors fast.
 - The **Nearest Neighbor Search** problem.

Nearest Neighbor Search

- Two-dimensional **kd-trees**
 - A data structure for answering nearest neighbor queries in $\mathbb{R}^2$
- kd-tree construction algorithm
 - Select the **x** or **y** dimension (alternating between the two)
 - Partition the space into two with a line passing from the median point
 - Repeat recursively in the two partitions as long as there are enough points

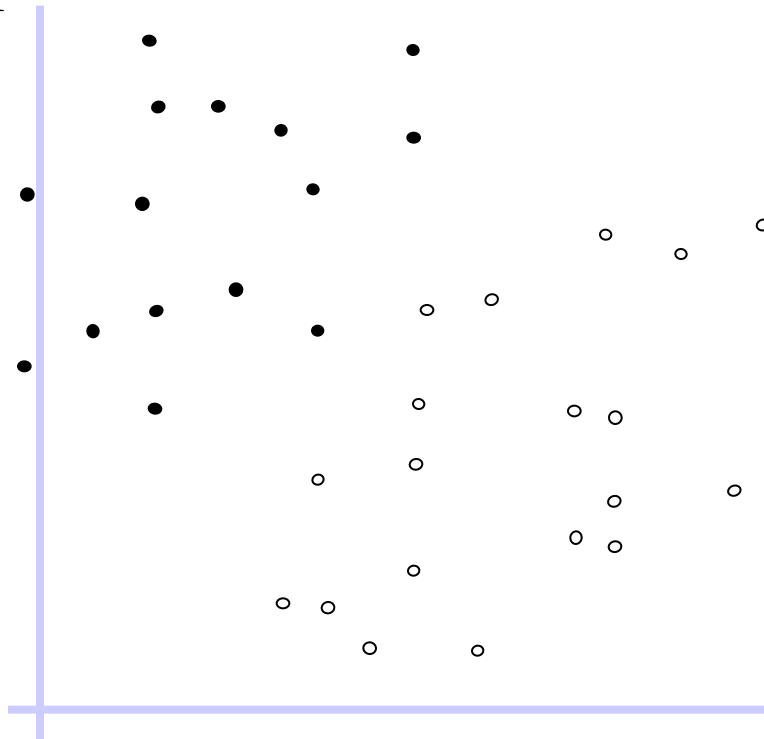
Support Vector Machines

Linear Classifiers



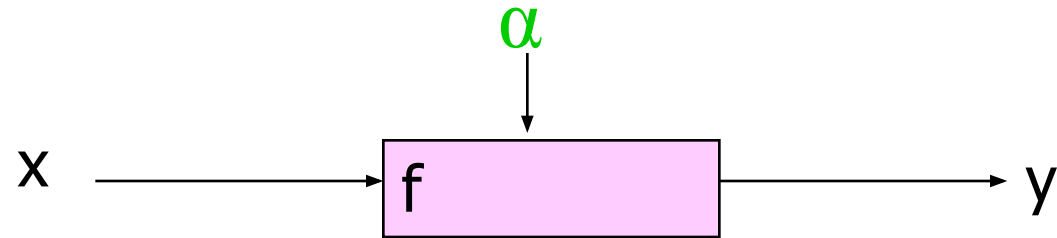
$$f(x, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

- denotes +1
- denotes -1

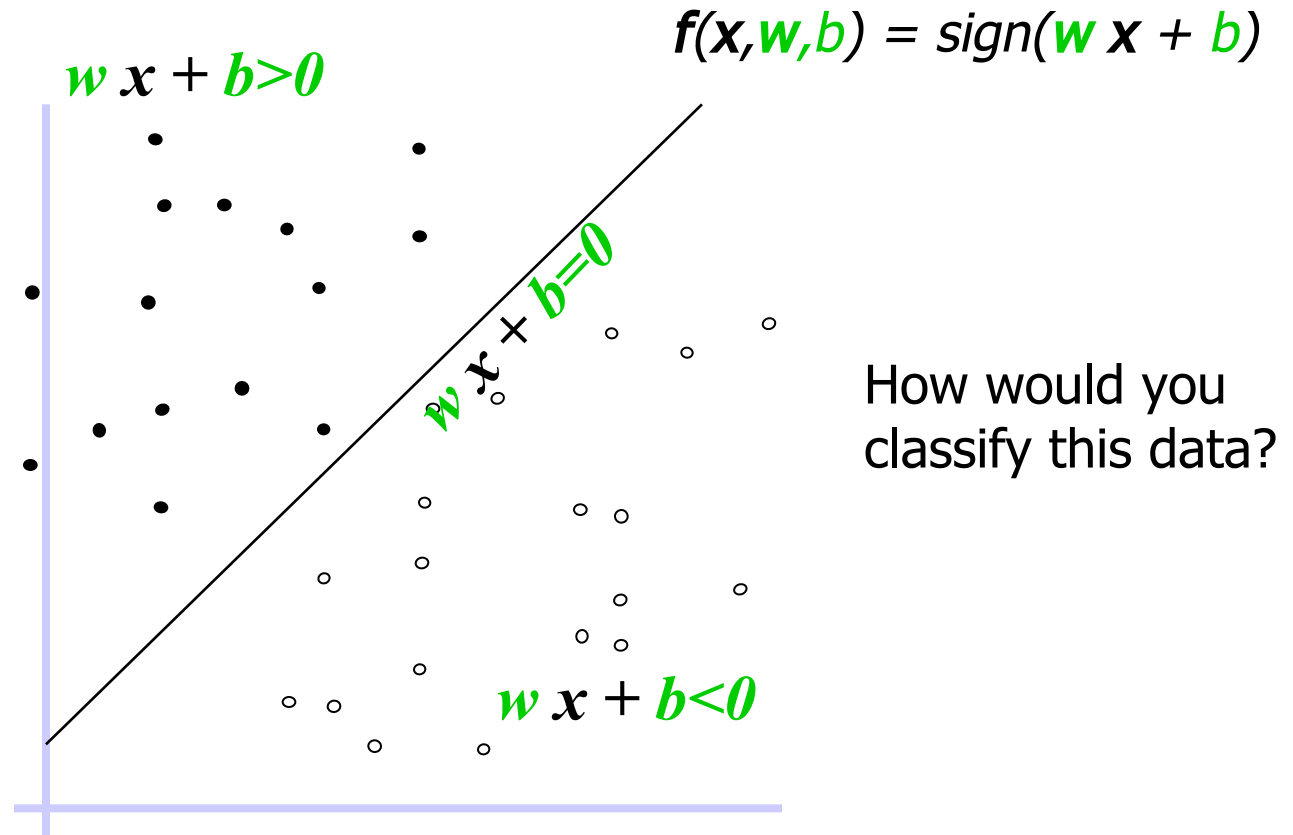


How would you classify this data?

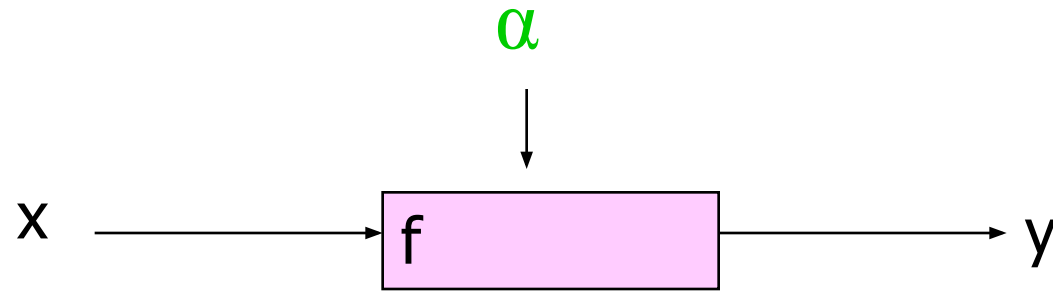
Linear Classifiers



- denotes +1
- denotes -1



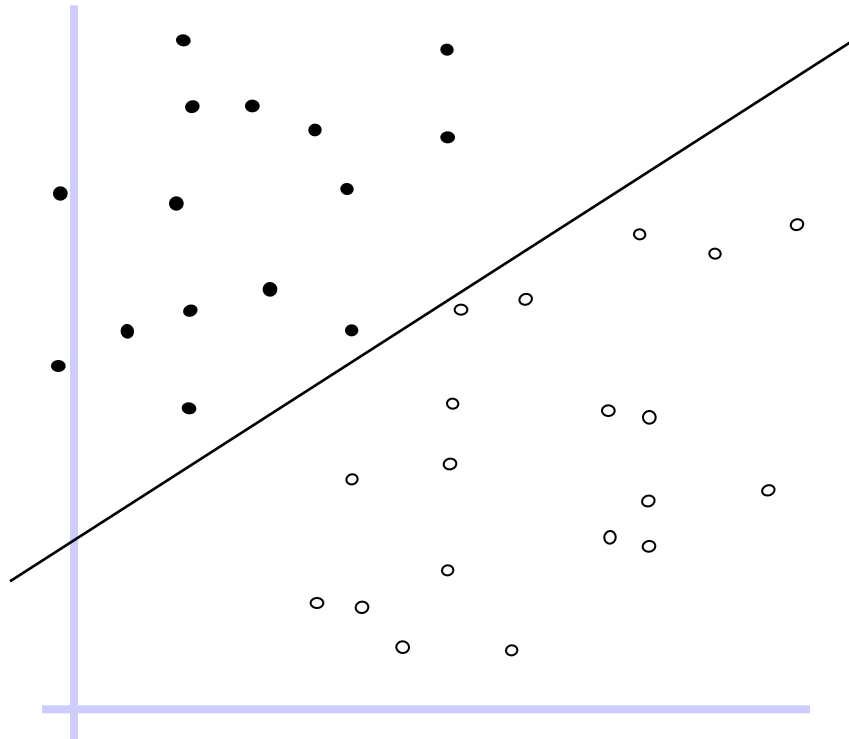
Linear Classifiers



$$f(x, \mathbf{w}, b) = \text{sign}(\mathbf{w} x + b)$$

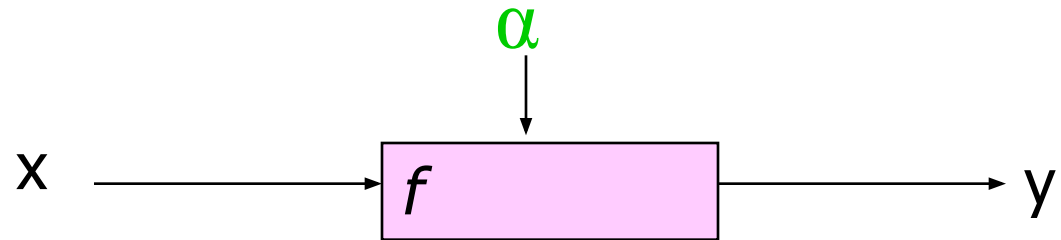
• denotes +1

○ denotes -1

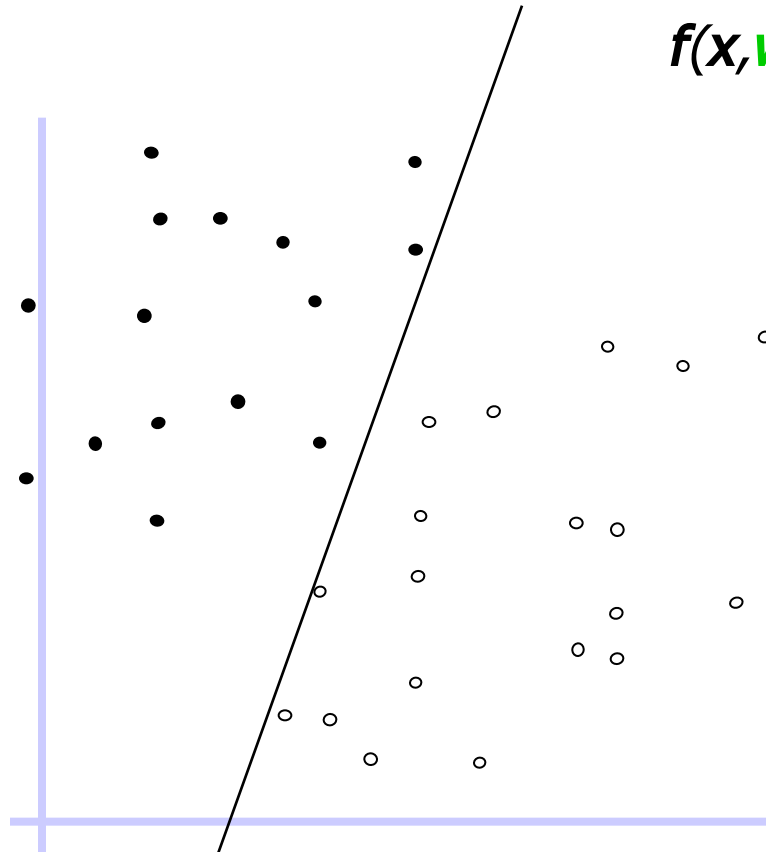


How would you classify this data?

Linear Classifiers



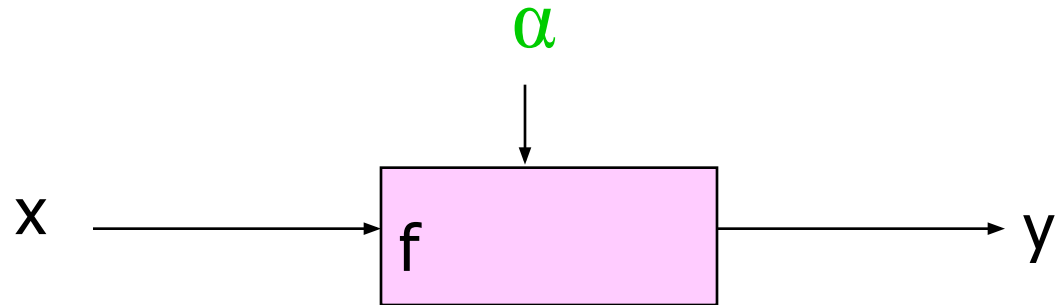
- denotes +1
- denotes -1



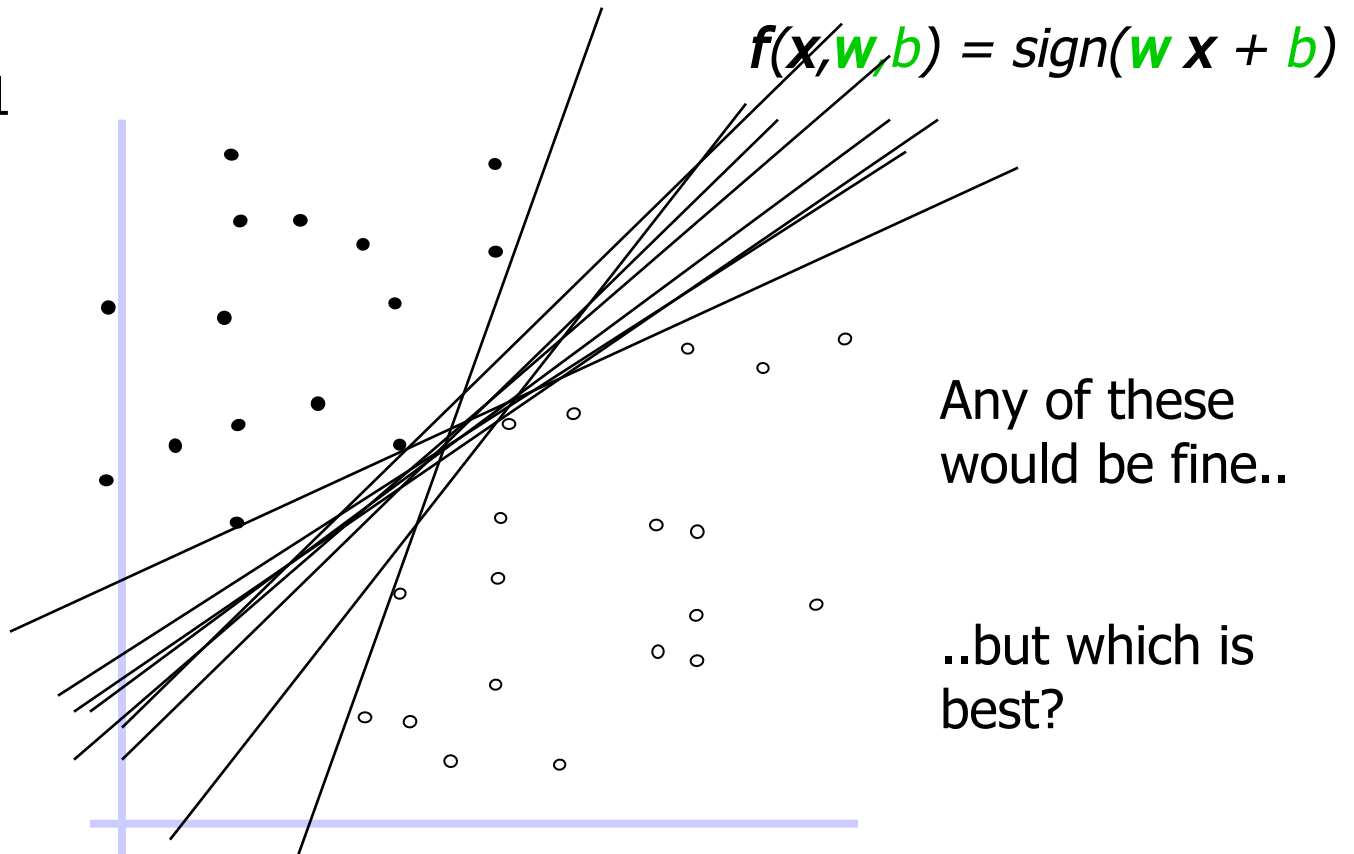
$$f(x, \mathbf{w}, b) = \text{sign}(\mathbf{w}x + b)$$

How would you classify this data?

Linear Classifiers



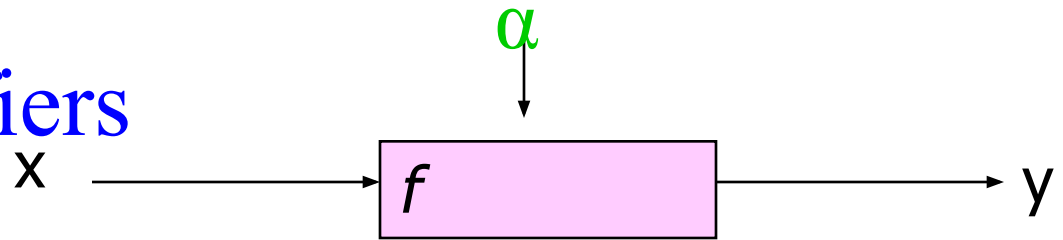
- denotes +1
- denotes -1



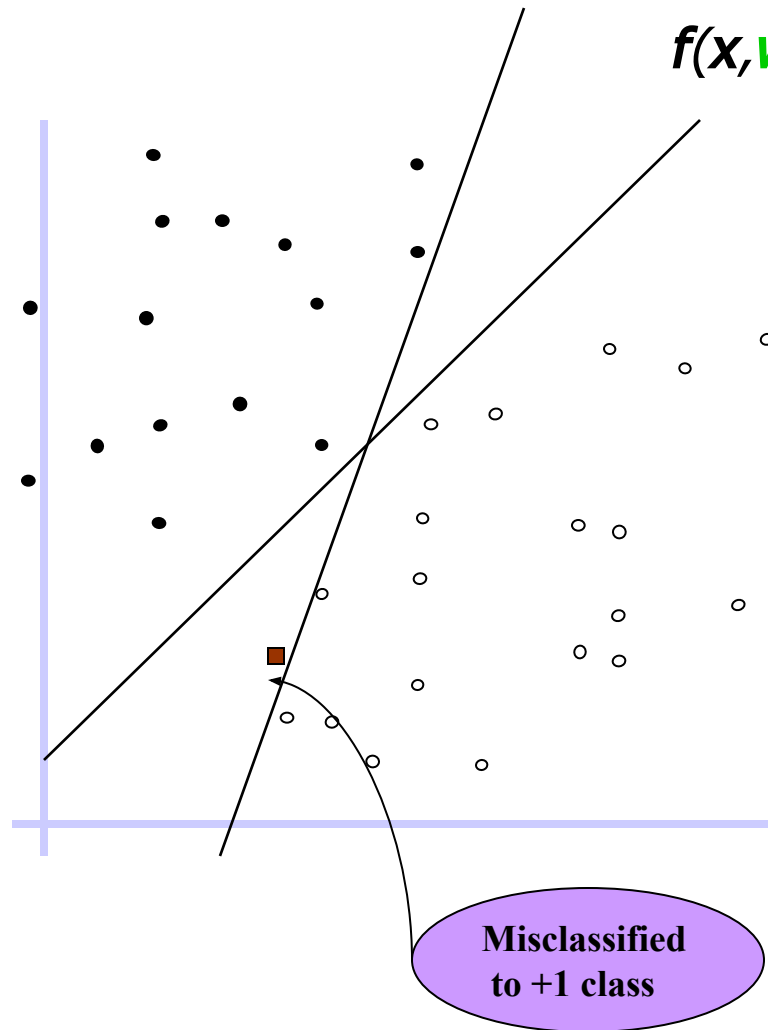
Any of these
would be fine..

..but which is
best?

Linear Classifiers



- denotes +1
- denotes -1

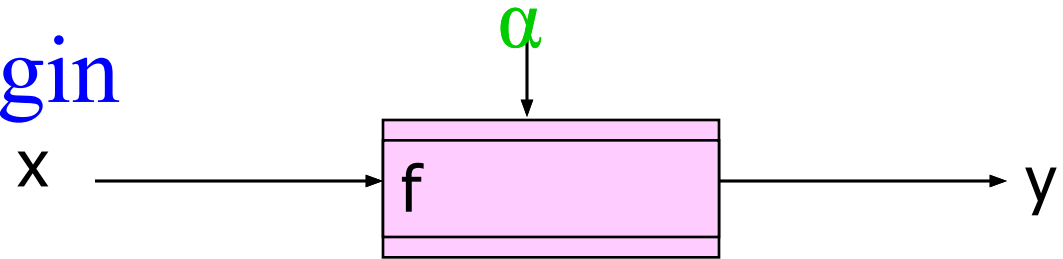


$$f(x, \mathbf{w}, b) = \text{sign}(\mathbf{w} x + b)$$

How would you classify this data?

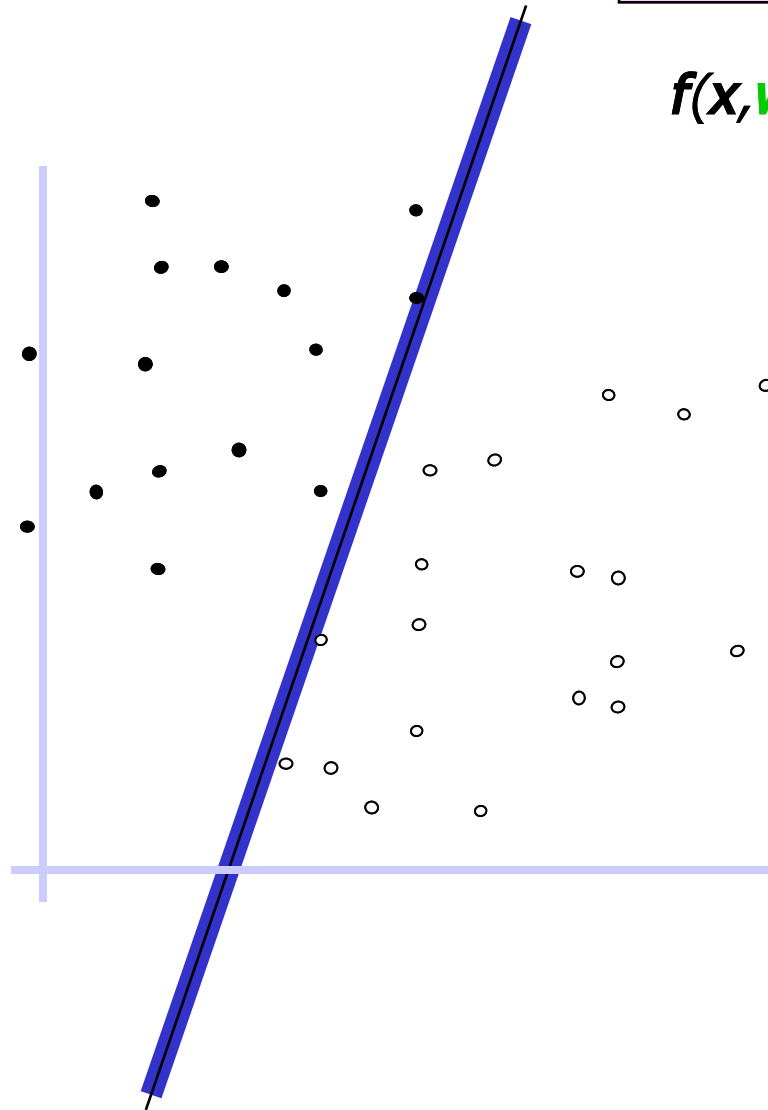
**Misclassified
to +1 class**

Classifier Margin



$$f(x, \mathbf{w}, b) = \text{sign}(\mathbf{w} x + b)$$

- denotes +1
- denotes -1



Define the **margin** of a linear classifier as the width that the boundary could be increased by before hitting a data point.

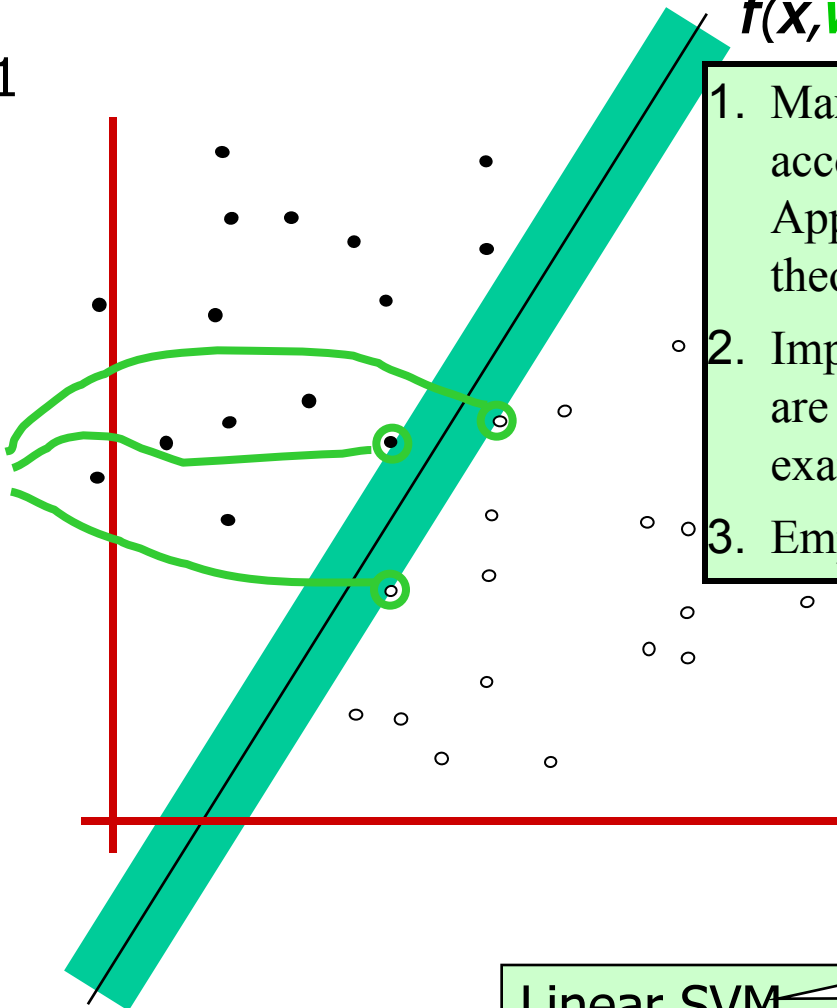
Maximum Margin



$$f(x, w, b) = \text{sign}(w x + b)$$

- denotes +1
- denotes -1

Support Vectors
are those
datapoints that
the margin
pushes up
against



1. Maximizing the margin is good according to intuition and Probably Approximately Correct (PAC) theory
- 2. Implies that only support vectors are important; other training examples are ignorable.
3. Empirically it works very well.

The **maximum margin linear classifier** is the linear classifier with the, um, maximum margin.

This is the simplest kind of SVM (Called an LSVM)

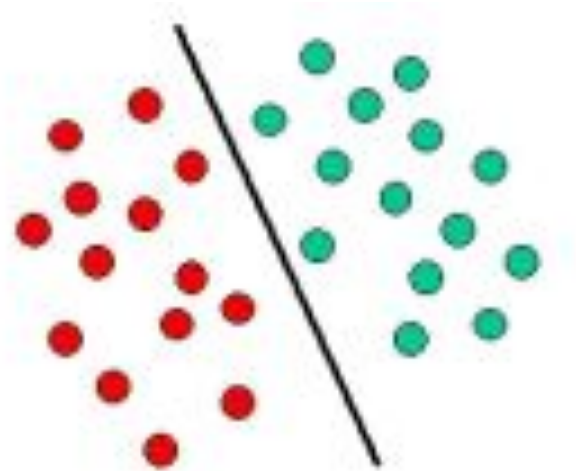
Linear SVM

Support Vector Machines

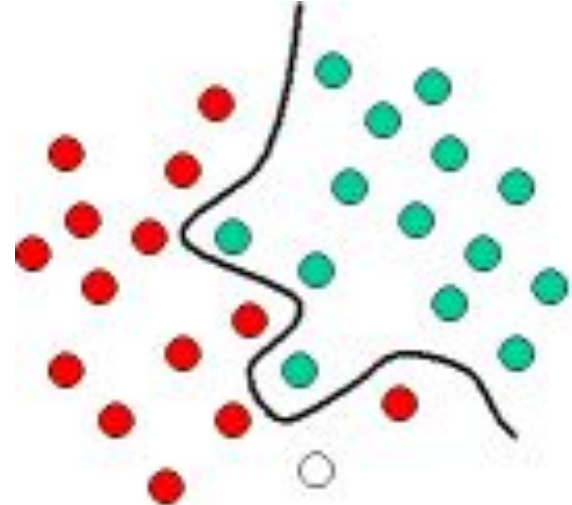
- In machine learning, **support vector machines** (SVMs, also **support vector networks**) are supervised learning models with associated learning algorithms that analyze data used for classification and regression analysis.
- More formally, a support vector machine constructs a hyperplane or set of hyperplanes in a high- or infinite-dimensional space, which can be used for classification, regression, or other tasks.
- Intuitively, a good separation is achieved by the hyperplane that has the largest distance to the nearest training-data point of any class (so-called **functional margin**), since in general the **larger the margin the lower the generalization error of the classifier**.

Support Vector Machines

- Support Vector Machines are based on the **concept of decision planes** that define **decision boundaries**. A decision plane is one that separates between a set of objects having different class memberships.
 - A schematic example is shown here. In this example, the objects belong either to class GREEN or RED.
- The separating line defines a boundary on the right side of which all objects are GREEN and to the left of which all objects are RED.
 - Any new object (white circle) falling to the right is labeled, i.e., classified, as GREEN (or classified as RED should it fall to the left of the separating line).

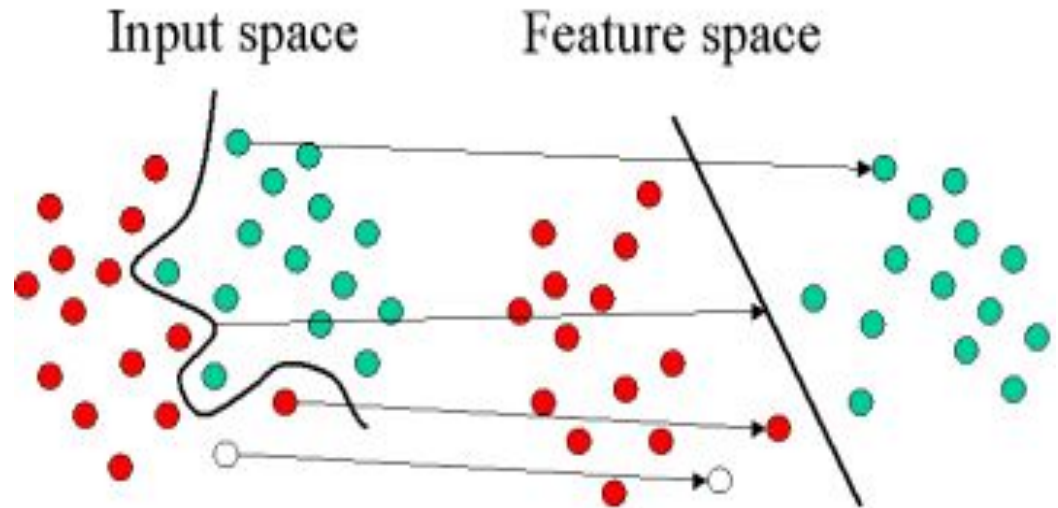


Support Vector Machines

- Previous is a **classic example of a linear classifier**, i.e., a classifier that separates a set of objects into their respective groups (GREEN and RED in this case) with a line.
 - Most classification tasks are not that simple
 - So, more complex structures are needed in order to make an optimal separation, i.e., correctly classify new objects (**test cases**) on the basis of the examples that are available (**train cases**).
- 
- Compared to the previous schematic, it is clear that a full separation of the GREEN and RED objects would require a curve (which is more complex than a line).
 - Classification tasks based on drawing separating lines to distinguish between objects of different class memberships are known as hyperplane classifiers.
 - **Support Vector Machines are particularly suited to handle such tasks.**

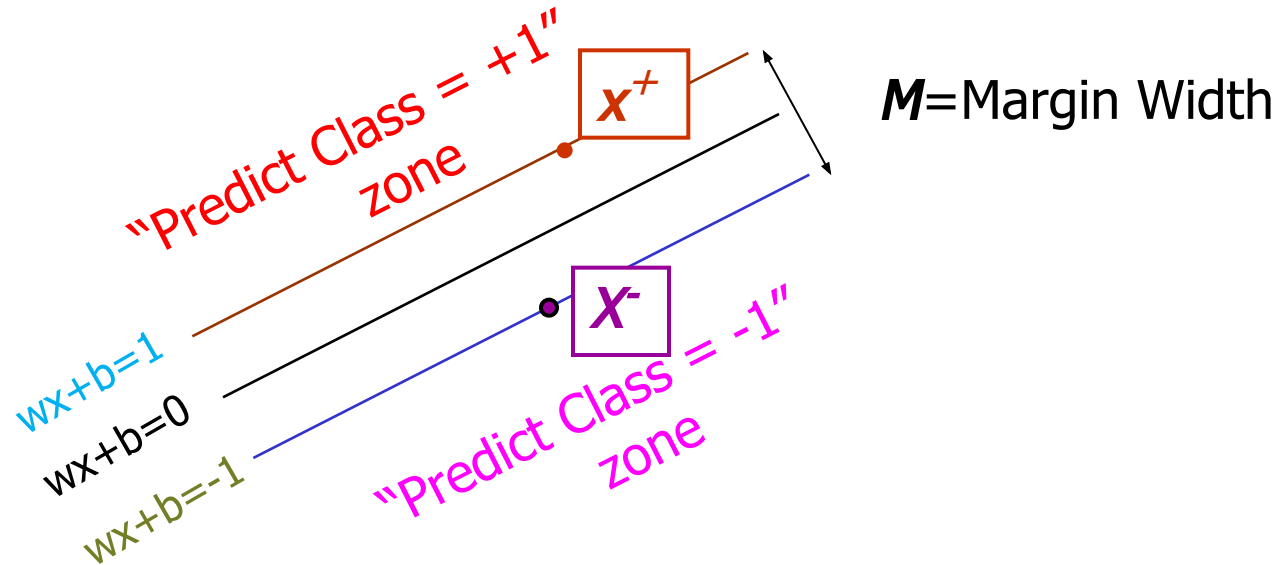
Support Vector Machines

- This shows the basic idea behind Support Vector Machines.
- Here we see the original objects (left side of the schematic) **mapped**, i.e., rearranged, using a set of mathematical functions, known as **kernels**.



- The **process of rearranging** the objects is known as **mapping** (**transformation**).
- Note that in this new setting, the mapped objects (right side of the schematic) is linearly separable and, thus, instead of constructing the complex curve (left schematic), all we have to do is to find an optimal line that can separate the GREEN and the RED objects.

Linear SVM Mathematically



What we know:

- $w \cdot x^+ + b = +1$
- $w \cdot x^- + b = -1$
- $w \cdot (x^+ - x^-) = 2$

$$M = \frac{(x^+ - x^-) \cdot w}{|w|} = \frac{2}{|w|}$$

Linear SVM Mathematically

■ Goal: 1) Correctly classify all training data

$$wx_i + b \geq 1 \quad \text{if } y_i = +1$$

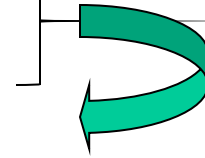
$$wx_i + b \leq -1 \quad \text{if } y_i = -1$$

$$y_i(wx_i + b) \geq 1 \quad \text{for all } i$$

2) Maximize the Margin

same as minimize

$$M = \frac{2}{|w|}$$
$$\frac{1}{2} w^t w$$



■ We can formulate a Quadratic Optimization Problem and solve for w and b

■ Minimize

$$\Phi(w) = \frac{1}{2} w^t w$$

subject to $y_i(wx_i + b) \geq 1 \quad \forall i$

Solving the Optimization Problem

Find $\mathbf{w}$ and b such that

$\Phi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w}$ is minimized;

and for all $\{(\mathbf{x}_i, y_i)\}: y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1$

- Need to optimize a *quadratic* function subject to *linear* constraints.
- Quadratic optimization problems are a well-known class of mathematical programming problems, and many (rather intricate) algorithms exist for solving them.
- The solution involves constructing a *dual problem* where a *Lagrange multiplier* α_i is associated with every constraint in the primary problem:

Find $\alpha_1 \dots \alpha_N$ such that

$\mathbf{Q}(\boldsymbol{\alpha}) = \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$ is maximized and

(1) $\sum \alpha_i y_i = 0$

(2) $\alpha_i \geq 0$ for all α_i

The Optimization Problem Solution

- The solution has the form:

$$\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i \quad b = y_k - \mathbf{w}^T \mathbf{x}_k \text{ for any } \mathbf{x}_k \text{ such that } \alpha_k \neq 0$$

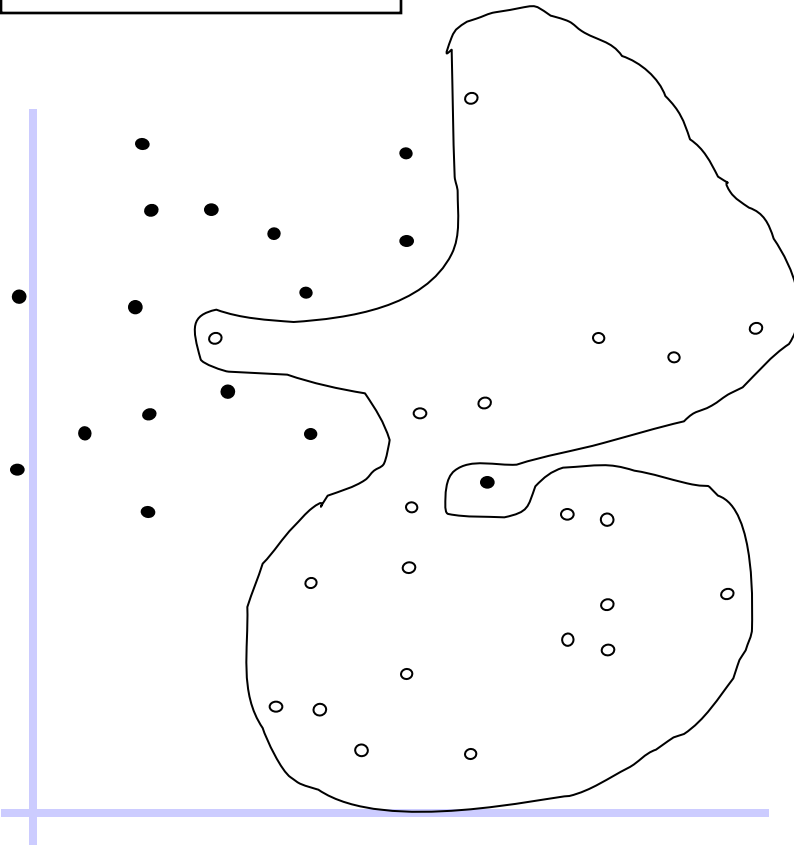
- Each non-zero α_i indicates that corresponding $\mathbf{x}_i$ is a support vector.
- Then the classifying function will have the form:

$$f(\mathbf{x}) = \sum_i \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b$$

- Notice that it relies on an *inner product* between the test point $\mathbf{x}$ and the support vectors $\mathbf{x}_i$ – we will return to this later.
- Also keep in mind that solving the optimization problem involved computing the inner products $\mathbf{x}_i^T \mathbf{x}_j$ between all pairs of training points.

Dataset with noise

- denotes +1
- denotes -1

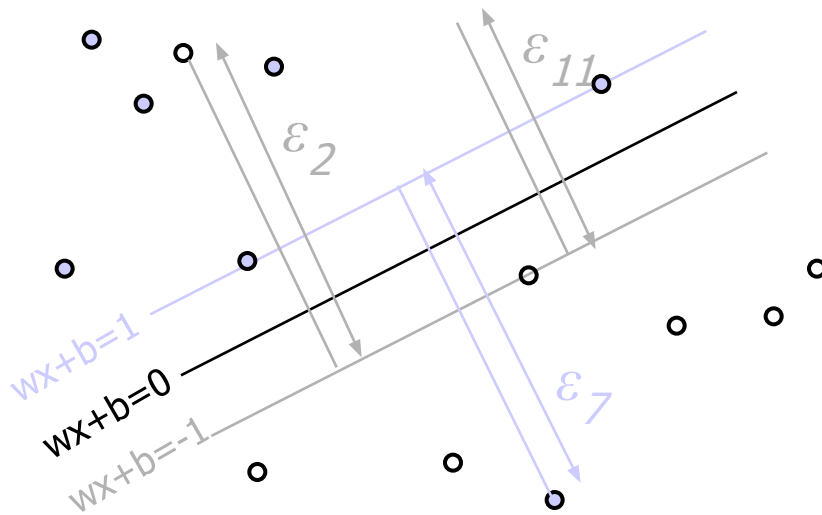


- **Hard Margin:** So far we require all data points be classified correctly
 - No training error
- **What if the training set is noisy?**
 - **Solution 1:** use very powerful kernels

OVERFITTING!

Soft Margin Classification

Slack variables ξ_i can be added to allow misclassification of difficult or noisy examples.



What should our quadratic optimization criterion be?

Minimize

$$\frac{1}{2} \mathbf{w} \cdot \mathbf{w} + C \sum_{k=1}^R \varepsilon_k$$

Hard Margin v.s. Soft Margin

- **The old formulation:**

Find $\mathbf{w}$ and b such that

$$\Phi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} \text{ is minimized and for all } \{(\mathbf{x}_i, y_i)\}$$
$$y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1$$

- **The new formulation incorporating slack variables:**

Find $\mathbf{w}$ and b such that

$$\Phi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum \xi_i \text{ is minimized and for all } \{(\mathbf{x}_i, y_i)\}$$
$$y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i \quad \text{and} \quad \xi_i \geq 0 \text{ for all } i$$

- **Parameter C can be viewed as a way to control overfitting.**

Linear SVMs: Overview

- The classifier is a *separating hyperplane*.
- Most “important” training points are support vectors; they define the hyperplane.
- Quadratic optimization algorithms can identify which training points $\mathbf{x}_i$ are support vectors with non-zero Lagrangian multipliers α_i .
- Both in the dual formulation of the problem and in the solution training points appear only inside dot products:

Find $\alpha_1 \dots \alpha_N$ such that

$Q(\alpha) = \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$ is maximized and

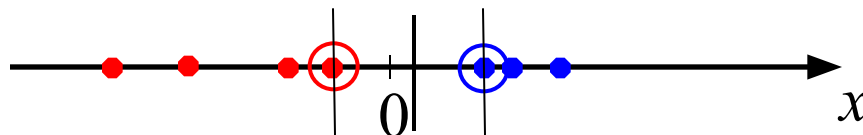
(1) $\sum \alpha_i y_i = 0$

(2) $0 \leq \alpha_i \leq C$ for all α_i

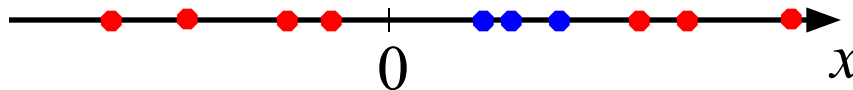
$$f(\mathbf{x}) = \sum \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b$$

Non-linear SVMs

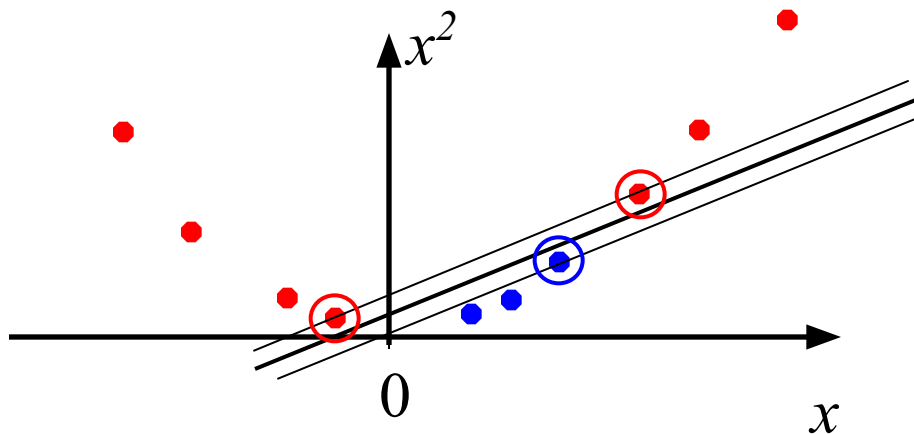
- Datasets that are linearly separable with some noise work out great:



- But what are we going to do if the dataset is just too hard?

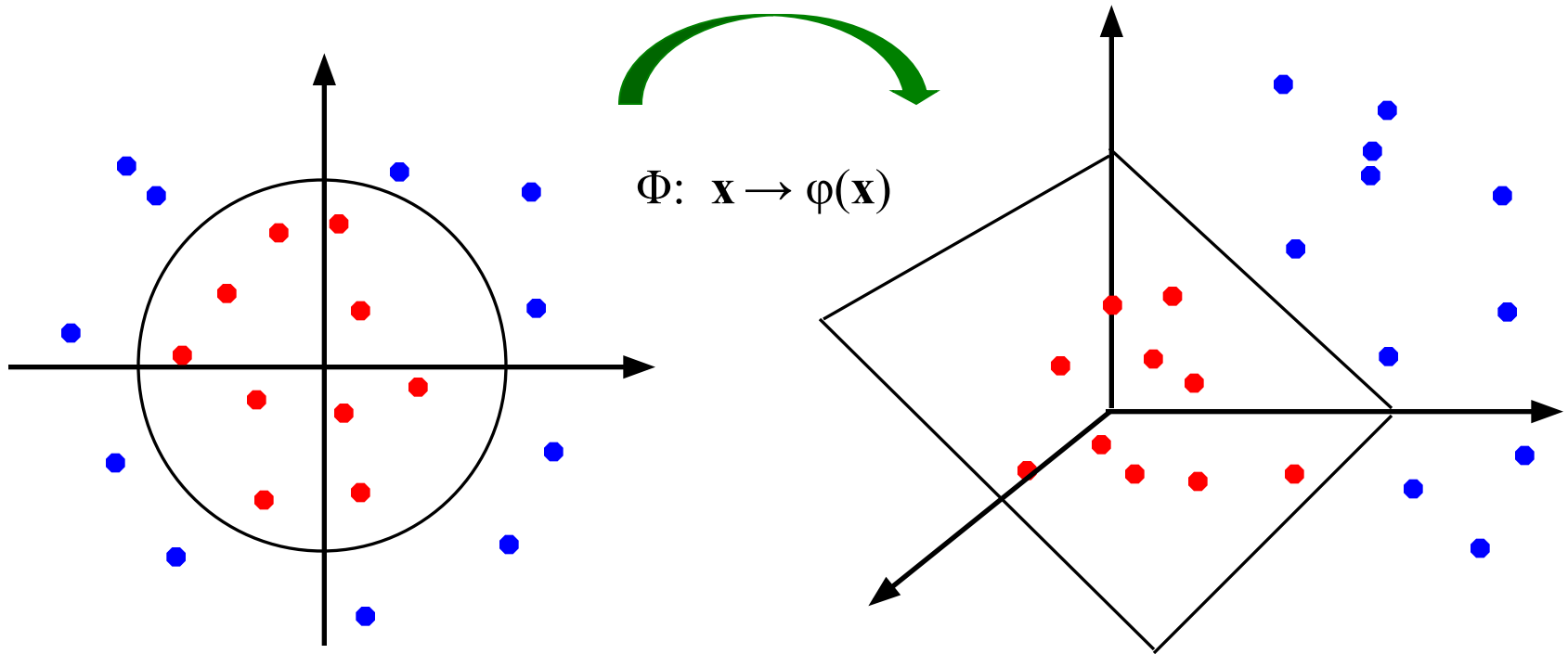


- How about... mapping data to a higher-dimensional space:



Non-linear SVMs: Feature spaces

- General idea: the original input space can always be mapped to some higher-dimensional feature space where the training set is separable:



The “Kernel Trick”

- The linear classifier relies on dot product between vectors $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
- If every data point is mapped into high-dimensional space via some transformation $\Phi: \mathbf{x} \rightarrow \phi(\mathbf{x})$, the dot product becomes:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

- A *kernel function* is some function that corresponds to an inner product in some expanded feature space.

- Example:

2-dimensional vectors $\mathbf{x} = [x_1 \ x_2]$; let $K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^T \mathbf{x}_j)^2$,

Need to show that $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$:

$$\begin{aligned} K(\mathbf{x}_i, \mathbf{x}_j) &= (1 + \mathbf{x}_i^T \mathbf{x}_j)^2 \\ &= 1 + x_{i1}^2 x_{j1}^2 + 2 x_{i1} x_{j1} x_{i2} x_{j2} + x_{i2}^2 x_{j2}^2 + 2 x_{i1} x_{j1} + 2 x_{i2} x_{j2} \\ &= [1 \ x_{i1}^2 \ \sqrt{2} x_{i1} x_{i2} \ x_{i2}^2 \ \sqrt{2} x_{i1} \ \sqrt{2} x_{i2}]^T [1 \ x_{j1}^2 \ \sqrt{2} x_{j1} x_{j2} \ x_{j2}^2 \ \sqrt{2} x_{j1} \ \sqrt{2} x_{j2}] \\ &= \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j), \quad \text{where } \phi(\mathbf{x}) = [1 \ x_1^2 \ \sqrt{2} x_1 x_2 \ x_2^2 \ \sqrt{2} x_1 \ \sqrt{2} x_2] \end{aligned}$$